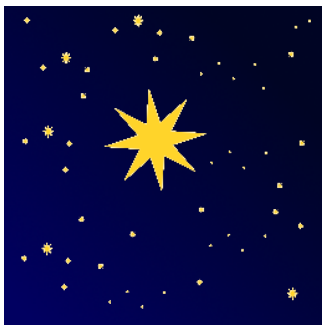


SightingDB

Once is happenstance. Twice is coincidence.
The third time it's enemy action.

— Auric Goldfinger, in Ian Fleming's *Goldfinger*, 1959

SightingDB does the counting.

Sebastien Tricaud

SightingDB 0.7

Contents

1 What this is **1**
- 1.1 What a sighting is . . . 1
- 1.2 Namespaces are paths . . . 2
- 1.3 Sightings, elsewhere . . . 2
  - 1.3.1 MISP . . . 2
  - 1.3.2 STIX 2.1 . . . 3
  - 1.3.3 Where this fits . . . 3
- 1.4 What it is not . . . 4
- 1.5 Where it sits . . . 4

2 Quick start **5**
- 2.1 Run it . . . 5
- 2.2 Write a sighting . . . 5
- 2.3 Read it back . . . 6
- 2.4 Write a lot of them at once . . . 6
- 2.5 Open the interface . . . 6
- 2.6 Try it from Python . . . 6
- 2.7 A galaxy, if you want one now . . . 7
- 2.8 What to read next . . . 7

3 Installing **8**
- 3.1 Build it . . . 8
- 3.2 Let it install itself . . . 8
- 3.3 Run it in a container . . . 9
- 3.4 Kubernetes . . . 9
- 3.5 Checking it started . . . 10

4 Configuration **11**
- 4.1 Yours, and the program's . . . 11
- 4.2 The minimum . . . 12
- 4.3 `[daemon]` . . . 12
- 4.4 `[storage]` . . . 13
- 4.5 `[acl]`, or `acl_file` . . . 14
- 4.6 `[galaxy]` . . . 14
- 4.7 `[stix]`, `[zmq]`, `[dns]` . . . 14
- 4.8 Changing it while it runs . . . 15

5 The API **16**
- 5.1 Writing . . . 16
  - 5.1.1 One sighting . . . 16
  - 5.1.2 Many at once . . . 17
  - 5.1.3 Checking a batch first . . . 17
- 5.2 Reading . . . 17
  - 5.2.1 One value . . . 17

5.2.2 With statistics . . . 18
5.2.3 A whole namespace . . . 18
5.2.4 Just the size of one . . . 18
5.2.5 Many values at once . . . 18
5.3 Deleting . . . 18
5.4 What the status codes mean . . . 19
5.5 Namespaces beginning with _ . . . 19
5.6 The route list . . . 19

6 Tags **20**
6.1 The shape of a tag set . . . 20
6.2 They merge . . . 20
6.3 The vocabulary the export reads . . . 21
6.4 Colours, and what a tag means . . . 21
6.4.1 Families . . . 21
6.4.2 What ships . . . 22
6.4.3 Tags found on values . . . 23
6.4.4 Typing one . . . 23
6.5 Where tags show up . . . 23

7 The Python client **24**
7.1 Connecting . . . 24
7.2 Writing . . . 24
7.3 Writing a lot . . . 25
7.4 Reading . . . 25
7.5 The rest . . . 26
7.6 Asynchronously . . . 26
7.7 Talking to several servers . . . 26
7.8 A worked example . . . 27

8 Galaxies **29**
8.1 Why more than one . . . 29
8.2 The three roles . . . 29
8.3 What a galaxy looks like . . . 30
8.4 Forwarding . . . 30
8.5 Counting, without inflating . . . 31
8.6 Consensus across a galaxy . . . 32
8.7 Catching up . . . 32
8.8 Cascades . . . 33
8.9 Keys are the boundary . . . 33
8.10 When nothing holds it . . . 34

9 Running a galaxy **35**
9.1 Build one in four containers . . . 35
9.2 Adding a node . . . 35
9.2.1 The node does not have to be up . . . 36
9.2.2 Give the new node peers of its own . . . 36
9.3 Changing one . . . 37
9.4 Taking one out of service . . . 37
9.5 Removing one . . . 38
9.6 What you cannot change over the API . . . 38
9.7 Watching it . . . 38

9.8 Keys across a galaxy . . . 38
9.9 A checklist . . . 39

10 The management interface **40**
10.1 Browsing . . . 40
10.2 Values . . . 40
10.3 One value . . . 41
10.4 Tags . . . 41
10.5 Galaxy . . . 41
10.6 Activity . . . 41
10.7 Keys . . . 45
10.8 Configuration . . . 46
10.9 Exporting . . . 46
10.10 Rejections . . . 46
10.11 A note on caching . . . 46

11 STIX and MISP **47**
11.1 Exporting STIX 2.1 . . . 47
11.1.1 Ids are derived, not minted . . . 47
11.1.2 What decides the pattern . . . 49
11.1.3 Headers say what you got . . . 49
11.1.4 A subtree, or several namespaces . . . 50
11.1.5 Configuring who you are . . . 50
11.2 Importing STIX . . . 50
11.3 SightingDB and MISP . . . 51
11.3.1 The connection in MISP . . . 52
11.3.2 What MISP shows . . . 53
11.3.3 Feeding it . . . 55
11.3.4 Pushing: what analysts record in MISP . . . 56
11.3.5 Publishing: MISP's attributes over ZMQ . . . 56
11.3.6 Checking each path . . . 58
11.4 A feed to try it with . . . 59

12 Operations **60**
12.1 Is it alive . . . 60
12.2 Where the data is . . . 60
12.3 What stays in memory . . . 60
12.4 Expiring data . . . 61
12.5 What was refused . . . 61
12.6 How busy each namespace is . . . 61
12.7 Certificates . . . 62
12.8 Logs . . . 62
12.9 Backing up . . . 62
12.10 Upgrading . . . 62
12.11 Things that have surprised people . . . 63

13 Reference **64**
13.1 Routes . . . 64
13.2 Query parameters . . . 65
13.3 Status codes . . . 66
13.4 Configuration at a glance . . . 66
13.5 The files . . . 67

13.6 Internal namespaces 67
13.7 The Python client 67
13.8 Where else to look 67

1

What this is

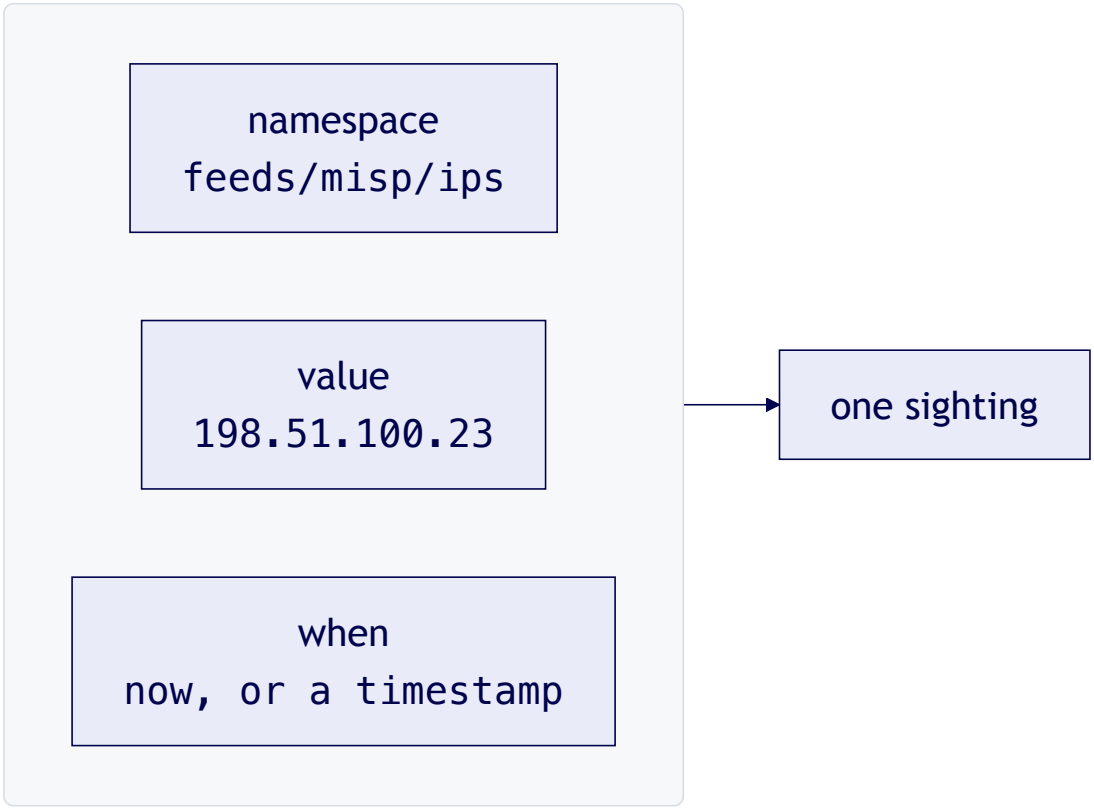
A sightings database answers one question: **how often have you seen this, and when?**

Not whether it is malicious. Not who owns it. Not what campaign it belongs to. Those are judgements, and there are good tools for keeping judgements. This keeps the observations the judgements are made from — and it keeps them at a volume and a speed that makes the count itself worth reading.

That is the whole idea. An address seen once last March is a different thing from an address seen four thousand times this morning, and from an address seen once a day for eighty days. Nothing about the address changed. What changed is what you know about it, and all you needed to know it was a counter.

1.1 What a sighting is

A sighting is a value, in a namespace, at a moment:



Write the same value again and the count goes up. That is the entire write path, and it is why this is fast: there is no document to re-index and no verdict to recompute.

What comes back when you read it:

Field	Meaning
`count`	How many times this namespace has seen this value.
`first_seen`, `last_seen`	The ends of the window it has been seen in.
`tags`	What the value *is*, as a set — see chapter 6.
`ttl`	When it stops being visible, if ever.
`consensus`	How many namespaces have seen it at all.
`stats`	The count broken down by hour, on request.

`consensus` is the one that is not obvious. It counts *namespaces*, not sightings: a value seen a thousand times in one feed has a consensus of one, and a value seen once each in six feeds has a consensus of six. The first is noisy; the second is corroborated. They are different questions and they deserve different numbers.

1.2 Namespaces are paths

A namespace is a path, and it browses like folders:

```
feeds/misp/ips
feeds/misp/domains
myorg/proxy/domains
watchlist/ips
```

There is no schema to declare. Writing to `feeds/misp/ips` creates it. The path is yours to organise: by source, by type, by team, by sensitivity. What it buys you is that access is granted by prefix — a key can be given `rw:feeds` and reach everything under it and nothing else — and that storage is managed by top-level namespace, so `feeds` can live in memory while `archive` lives on disk.

1.3 Sightings, elsewhere

The word is not ours, and it is worth knowing what the two standards that use it mean by it, because SightingDB sits deliberately between them.

1.3.1 MISP

MISP's definition is a person speaking:

> Sighting is a way for a user to say that they have seen or notice an attribute and confirm its validity.
> — MISP's documentation

A MISP sighting is therefore partly a **judgement**. There are three kinds — a *true positive*, a *false positive*, and an *expiration date* saying an attribute has stopped being relevant — and they feed MISP's scoring and decaying models. An analyst saying "yes, we saw this, and it was real" is exactly the signal those models need.

1.3.2 STIX 2.1

STIX models a sighting as a relationship between an indicator and whoever saw it. What distinguishes it from every other relationship is the three properties it alone carries:

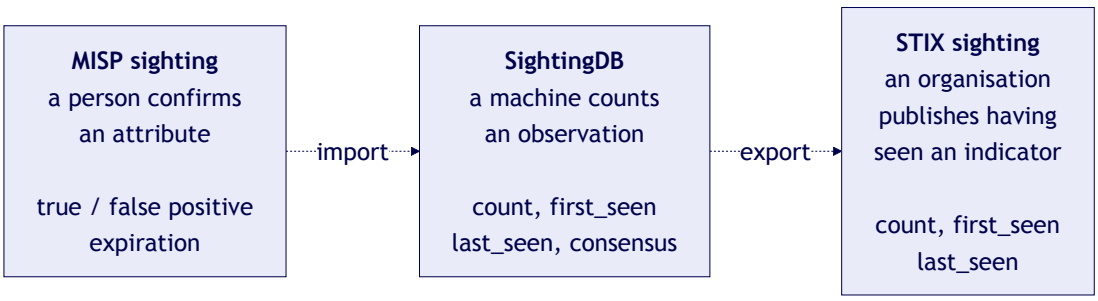
> Sighting contains unique properties like `count`, `first_seen`, and `last_seen` that convey when a SDO was seen within a particular timeframe as well as the number of times this SDO was seen.
> — OASIS, *Sighting of an Indicator*

In the OASIS example, one organisation publishes an indicator for a malicious URL and another, seeing it on their own network, publishes a sighting of it:

```
{
  "type": "sighting",
  "spec_version": "2.1",
  "id": "sighting--ee20065d-2555-424f-ad9e-0f8428623c75",
  "count": 1,
  "first_seen": "2017-02-27T21:37:11.213Z",
  "last_seen": "2017-02-27T21:37:11.213Z",
  "sighting_of_ref": "indicator--9299f726-ce06-492e-8472-2b52ccb53191",
  "where_sighted_refs": ["identity--..."]
}
```

1.3.3 Where this fits

Look again at what STIX says is special about a sighting — `count`, `first_seen`, `last_seen` — and then at what SightingDB stores. They are the same three fields. That is not a coincidence or a convenience: **this is a database of exactly the thing a STIX sighting is**, which is why the export in chapter 11 is a mapping rather than a translation.



The difference from MISP is the one worth holding on to. A MISP sighting is a **judgement** made a few times by people; a SightingDB sighting is an **observation** recorded continuously by machines, with no view on whether the thing is good or bad. That is why there is no "false

positive" here: nothing is being asserted that could be false. The value was seen, four thousand times, and what that means is yours to decide.

The two are complements rather than rivals. A MISP attribute whose decaying model says it has gone stale, and a SightingDB count that says your proxy has seen it eleven times this morning, are both true and interesting at once.

1.4 What it is not

It is not a threat intelligence platform. It holds no verdicts, no relationships, no cases, no workflow. It will happily tell you that you have seen `198.51.100.23` four thousand times and take no view on whether that is good or bad.

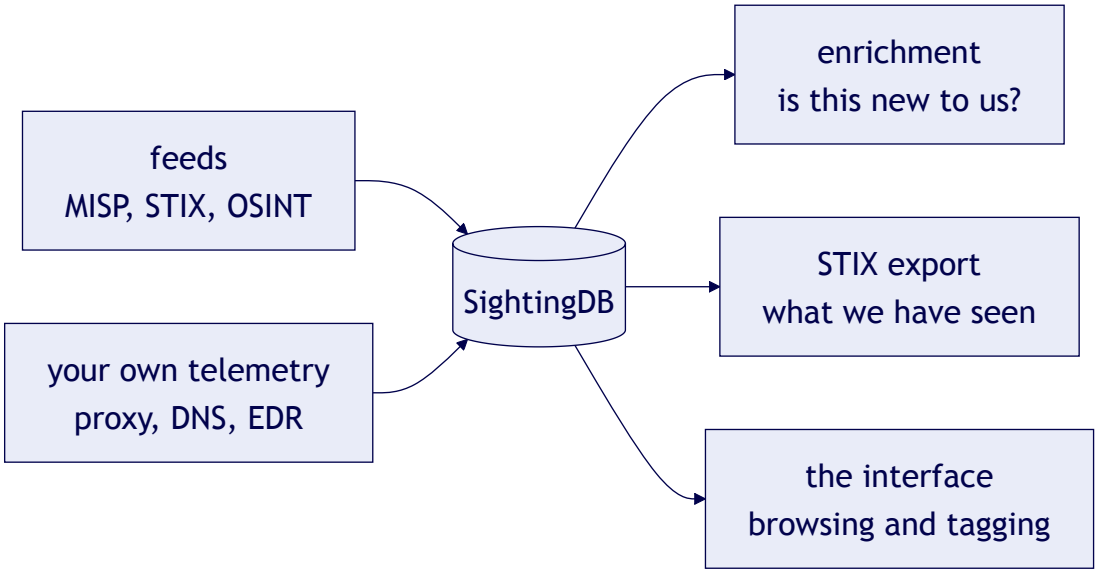
It is not a log store. A log line is an event with a shape; a sighting is a counter with a window. If you need to answer "what happened at 14:32", keep your logs. If you need to answer "is this the first time?", in the time it takes to answer an API call, this is the thing.

It is not a graph. Values do not point at each other.

> Those are not gaps to be filled in later. They are the reason a write is a counter increment and a read is a map lookup, which is what makes the numbers above cheap enough to ask for on every event you process.

1.5 Where it sits

Most deployments put it behind whatever is already producing observations — a proxy log pipeline, a MISP feed, a sensor — and in front of whatever is asking questions.



The rest of this book is how to run that, starting with the shortest possible version of it.

2

Quick start

Five minutes, one container, no configuration.

2.1 Run it

From a clone of the repository:

```
docker compose -f docker/docker-compose.yml up --build
```

That builds the image from the source in your working tree and starts one server on port 9999 with the API key `changeme`.

> There is no published image to pull. SightingDB is small and builds in a couple of minutes, and an image you built from the source in front of you is one fewer thing to trust.

If you would rather not use Docker, chapter 3 builds it directly; everything below is the same either way.

2.2 Write a sighting

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/w/feeds/ips?val=198.51.100.23'
```

```
{"message":"ok","count":1,"new":true}
```

Run it again. And again.

```
{"message":"ok","count":3,"new":false}
```

`new` is whether this namespace had ever seen the value before, which is the one thing you cannot work out from the count afterwards.

2.3 Read it back

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/r/feeds/ips?val=198.51.100.23&noshadow'
```

```
{"value":"198.51.100.23","first_seen":1791668207,"last_seen":1791668230,
 "count":3,"tags":"","ttl":0,"consensus":1}
```

`noshadow` is there because **a read is itself an observation**. Without it, SightingDB records that somebody asked — in a parallel `_shadow/` namespace — which is how you find out which indicators your analysts keep looking up. When you are reading your own data in a loop, say `noshadow` and it records nothing.

2.4 Write a lot of them at once

One sighting per HTTP request is fine for enrichment, and wrong for ingest:

```
curl -H 'Authorization: changeme' -H 'Content-Type: application/json' \
  -X POST http://localhost:9999/wb -d '{"items":[
    {"namespace":"feeds/ips","value":"198.51.100.23","tags":"tlp:green"},
    {"namespace":"feeds/ips","value":"203.0.113.7"},
    {"namespace":"feeds/domains","value":"login.phish.example"}
  ]}'
```

Every item gets its own result, so one bad value does not discard the batch:

```
{"message":"ok","written":3,"items":[
  {"index":0,"namespace":"feeds/ips","value":"198.51.100.23","status":"ok",
   "count":4,"new":false}, ...]}
```

2.5 Open the interface

http://localhost:9999/_management, with the key `changeme`.

2.6 Try it from Python

```
pip install sightingdb-client
```

```
from sightingdb import SightingDB

db = SightingDB(url="http://localhost:9999", apikey="changeme")

db.write("feeds/ips", "198.51.100.23")
print(db.read("feeds/ips", "198.51.100.23", shadow=False).count)
```

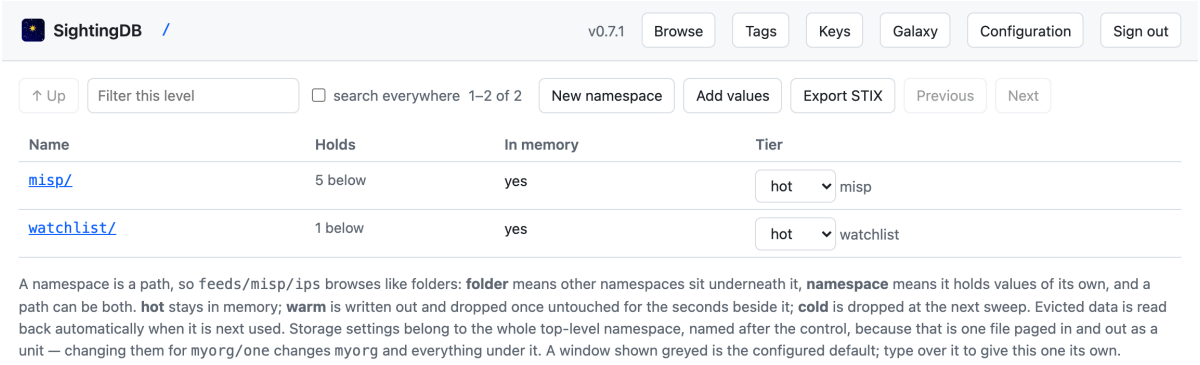


Figure 2.1: The namespace tree. A path is a folder and a namespace at once.

```
print(db.exists("feeds/ips", "203.0.113.99"))
```

Chapter 7 is the rest of it.

2.7 A galaxy, if you want one now

One server is a node. Several servers behind a load balancer is a *galaxy*, and there is a four-container sample of one:

```
docker compose -f doc/docker/docker-compose.yml up --build
```

That starts a load balancer on 9999 and three nodes — two full mirrors and one holding a single namespace — and loads them with real indicators from CIRCL's public OSINT feed. Chapter 8 explains what it is doing; `doc/docker/README.md` lists things to try.

2.8 What to read next

- **Chapter 3** to install it properly, with TLS and a service.
- **Chapter 4** for the configuration files.
- **Chapter 5** for the rest of the API.
- **Chapter 8** if you are here because one server is not enough.

3

Installing

Three ways, in rising order of permanence: build it and run it, let `--setup` install it, or run it in a container.

3.1 Build it

SightingDB is a single Rust binary with no runtime dependencies beyond OpenSSL.

```
git clone https://github.com/stricaud/sightingdb
cd sightingdb
cargo build --release
./target/release/sightingdb --help
```

To run it without installing anything:

```
./target/release/sightingdb -c etc/sightingdb.toml
```

`etc/sightingdb.toml` in the repository is a working configuration and is mostly comments explaining itself. **Read the `dbdir` line before you run it**: it points at `/var/lib/sighting`, which is where a real installation keeps its data, and is probably not what you want for a first look.

3.2 Let it install itself

```
sudo sightingdb --setup
```

This asks what the server is for and then writes everything: directories, a configuration with comments, an API key, a self-signed certificate, and a service for systemd or launchd. It asks before changing anything and tells you what it did.

The first question is the one that matters:

```
  1) standalone          stores everything; knows no peers
  2) node                stores namespaces; other servers forward to it
  3) router              stores nothing of its own; forwards to its peers
  4) both                stores some namespaces and forwards the rest
```

Chapter 8 is what those mean. If you are installing one server, it is 1.

What it writes, and which of them it will rewrite later:

File	Who owns it
`sightingdb.toml`	**You.** Comment-rich, hand-maintained, never rewritten.
`acl.toml`	The program. Rewritten when a key is saved.
`tiers.toml`	The program. Rewritten when a storage tier changes.
`tags.toml`	The program. Rewritten when a tag's colour changes.
`peers.toml`	The program. Rewritten when a peer is added or changed.

That split is the thing to understand about configuring SightingDB, and chapter 4 is about it.

> `--setup` writes a **self-signed** certificate. It is there so that the first thing you run is encrypted rather than plaintext, not because it is good enough for a galaxy that crosses a network you do not control. Replace it, or terminate TLS in front.

3.3 Run it in a container

```
docker compose -f docker/docker-compose.yml up --build
```

The image builds from your working tree, runs as a non-root user at a fixed uid, and keeps its database on a named volume. The configuration baked in is deliberately small — mount your own over `/etc/sightingdb/sightingdb.toml` to change it, which is what the galaxy sample in `doc/docker` does.

Two environment variables matter:

Variable	Effect
`SIGHTINGDB_APIKEY`	The key, granted full access. Read by the daemon itself, so it is not visible in `ps`.
`SIGHTINGDB_CONFIG`	Which configuration to read.

3.4 Kubernetes

```
make deploy        # into the cluster kubectl already points at
make dev           # into a kind cluster this creates for the purpose
```

Both build the image from your tree, install the Helm chart, wait for the pod and write a sighting through it to prove it works. **They refuse to touch a cluster that is not local** — they check the context against a list of local ones and stop otherwise, because a `kubectl config use-context`

away from production should not be one keystroke away from `make deploy`.

`make port-forward` and `make admin-key` then get you into the interface.

3.5 Checking it started

```
curl -fsS http://localhost:9999/health
```

```
{"status":"ok","version":"0.7.1","uptime_seconds":12,
 "resident_shards":2,"shards":2,"catching_up":false}
```

`/health` needs no key, which is what makes it usable as a container healthcheck and a load balancer probe. Everything else needs one.

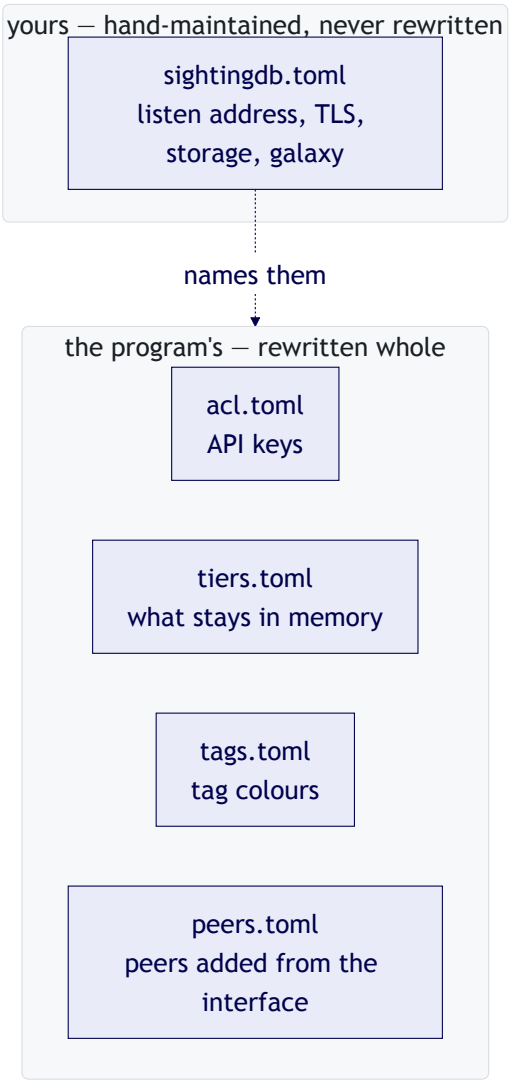
If it did not start, it said why and exited; look at `log_err` from the configuration, or the container's logs. A configuration that will not parse names the line.

4

Configuration

Everything is TOML, and there are five files. Knowing which of them is yours and which belongs to the program is most of what there is to know.

4.1 Yours, and the program's



`sightingdb.toml` is comment-rich and the program does not write to it. The other four it rewrites, wholesale, whenever you change the matching thing in the management interface.

That is why they are separate files. A program that rewrote the main configuration would destroy the comments in it, and those comments are the documentation you actually read at three in the morning. The cost is that **anything set directly in `sightingdb.toml` is read-only in the interface** — which it says, rather than appearing to accept a change that will not last.

If you do not name one of the four, the matching thing still works; it is simply not editable. No `tags_file` means the standard colours and a Tags page that says it cannot save. No `peers_file` means the peers in `[galaxy] peers` and a Galaxy page that cannot add one.

4.2 The minimum

```toml
[daemon]
listen_ip = "127.0.0.1"
listen_port = 9999
authenticate = true
ssl = false
dbdir = "/var/lib/sightingdb"
acl_file = "/var/lib/sightingdb/acl.toml"
```

That is a working server. Everything below has a default.

4.3 `[daemon]`

Setting	Default	What it does
`enabled`	`true`	`false` runs DNS or ZMQ ingest with no HTTP API.
`listen_ip`, `listen_port`	`127.0.0.1:9999`	Where the API listens.
`authenticate`	`true`	`false` disables API keys entirely. For a laptop.
`daemonize`	`false`	Fork into the background. Leave off in a container.
`ssl`, `ssl_cert`, `ssl_key`	off	Terminate TLS here.
`dbdir`	none	Where snapshots live. **Without it nothing survives a restart.**
`snapshot_interval`	300	Seconds between writing the database out.
`sweep_interval`	60	Seconds between expiring TTLs and evicting cold shards.
`stats_retention`	720	Hourly buckets kept per value. 720 is 30 days.
`shadow_ttl`	2592000	How long a record of "somebody searched for this" lasts.

Setting	Default	What it does
`post_limit`	2.5 GB	The largest bulk request accepted.
`rejection_log`	`1000`	How many refused values to keep for the interface.
`activity`	`true`	Count reads and writes per top-level namespace for the Activity view. `false` makes counting free.
`node_id`	`"local"`	**This server's name in its own counters.**
`acl_file`, `tags_file`	none	The machine-owned files above.
`log_out`, `log_err`	stdout	Where the logs go.

`node_id` is the one to get right before a server joins a galaxy. Sightings are counted per node, so two servers sharing a name each take the other's contributions for their own, and the merge that should reconcile them discards one side instead. Give every server its own.

4.4 [storage]

```toml
[storage]
default_tier = "hot"
warm_idle = 3600
tiers_file = "/var/lib/sightingdb/tiers.toml"
namespaces = ["feeds", "myorg"]
```

`namespaces` is what this server stores. Absent means everything. An empty list makes it a router, which stores nothing of its own and forwards — see chapter 8. Prefixes match whole path segments, so `feeds` holds `feeds/misp/ips` and never `feeds-internal`.

The three tiers decide what sits in memory:

Tier	Behaviour
`hot`	Stays in memory. The default.
`warm`	Written out and dropped after `warm_idle` seconds untouched.
`cold`	Dropped at the next sweep.

Evicted data is not gone — the shard is read back when it is next used, so `cold` costs one load per burst of activity rather than one per operation. Tiers apply to a whole top-level namespace, because that is one file paged in and out as a unit.

4.5 [acl], or acl_file

```
[acl]
"changeme" = "rw, admin"
"analyst"  = "r"
"feed-misp" = "rw:feeds/misp"
"lb-to-us" = "rw:feeds"
```

A grant is `r`, `w` or `rw`, each optionally followed by `:<prefix>`, plus `admin` for the management interface. Without a prefix the grant is global. Access is allow-only: a key is denied unless one of its grants both covers the namespace and carries the permission.

Set `acl_file` instead if you want the interface to manage keys. The two together is an error worth avoiding: the file wins, and an `acl_file` that does not exist yet starts *empty*, discarding an inline `[acl]` table. Use one or the other.

Out-of-reach namespaces answer `404` rather than `403` throughout, so that browsing cannot be used to enumerate what a key may not see.

4.6 [galaxy]

The whole of chapter 8. In short:

```
[galaxy]
peers_file = "/var/lib/sightingdb/peers.toml"
max_hops = 4
health_interval = 30
sync_interval = 300
reconcile_interval = 3600
gossip_interval = 600
peers = [
  { url = "https://node-a.example:9999", key = "..." },
  { url = "https://node-b.example:9999", key = "...", namespaces = ["feeds"] },
  { url = "https://spare.example:9999", key = "...", enabled = false },
]
```

A peer carries the key **this** server authenticates to it with, which is the bound on what it may do there, and what that peer holds, so a request can be sent where it can be served.

4.7 [stix], [zmq], [dns]

Three optional sections, each with a chapter or a section of its own:

- `[stix]` — who exports are published as, and which observable type a namespace holds by default. Chapter 11.
- `[zmq]` — subscribe to a MISP publisher and write what it sends. Chapter 11.
- `[dns]` — answer sightings over DNS, DNSBL-style, for tools that speak no HTTP. It **bypasses the ACL**, which is why its defaults are locked down.

4.8 Changing it while it runs

Change	Needs a restart?
API keys	No, if `acl_file` is set.
Storage tiers	No, if `tiers_file` is set.
Tag colours	No, if `tags_file` is set.
Galaxy peers	No, if `peers_file` is set.
Anything else	Yes.

The last row is honest rather than apologetic: the listen address, TLS, the DNS zone and the storage policy all need sockets rebound or the database rebuilt, so “editable at runtime” would mean “applied at the next restart” anyway.

5

The API

Every route takes the key in an `Authorization` header. There is no scheme prefix — the key is the whole value.

```
curl -H 'Authorization: changeme' ...
```

`doc/routes.md` in the repository is the exhaustive version of this chapter, with every status code. This is the part you will use.

5.1 Writing

5.1.1 One sighting

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/w/feeds/misp/ips?val=198.51.100.23'
```

```
{"message":"ok","count":4,"new":false}
```

The namespace is the path. Everything after `/w/` up to the query string is it, so it can be as deep as you like.

Optional parameters:

Parameter	Effect
`timestamp=`	Record it as seen then, in Unix seconds, rather than now.
`ttl=`	Seconds until it stops being visible.
`tags=`	Tags to **merge** into whatever it already carries.

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/w/feeds/misp/ips?val=198.51.100.23&tags=tlp:green,stix-type:ipv4-addr'
```

Writes merge tags rather than replacing them, so two feeds that each know something different about a value both contribute. Taking a tag *off* is the management interface's job — chapter 6.

5.1.2 Many at once

`POST /wb` is the ingest path, and the one to use for anything over a handful.

```
curl -H 'Authorization: changeme' -H 'Content-Type: application/json' \
  -X POST http://localhost:9999/wb -d '{"items":[
    {"namespace":"feeds/misp/ips","value":"198.51.100.23","tags":"tlp:green"},
    {"namespace":"feeds/misp/ips","value":"203.0.113.7","timestamp":1791668207},
    {"namespace":"feeds/misp/domains","value":"login.phish.example","ttl":86400}
  ]}'
```

Every item is answered separately:

```
{"message":"ok","written":3,"items":[
  {"index":0,"namespace":"feeds/misp/ips","value":"198.51.100.23",
   "status":"ok","count":5,"new":false},
  ...]}
```

`message` is `ok`, `partial` or `failed`. A batch where some items were refused and others written is `partial` with HTTP 200 — **the ones that worked are stored**, which is the point of per-item results. A batch that wrote nothing because every item was refused by the ACL is `403`; one that wrote nothing for mixed reasons is `400`.

5.1.3 Checking a batch first

`POST /vwb` takes the same body and writes nothing:

```
curl -H 'Authorization: changeme' -H 'Content-Type: application/json' \
  -X POST http://localhost:9999/vwb -d '{"items":[...]}'
```

It answers with the same per-item shape, saying which items *would* be accepted. It deliberately does not read the database, which is what lets it promise that a passing item would be written rather than that it looks plausible.

5.2 Reading

5.2.1 One value

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/r/feeds/misp/ips?val=198.51.100.23&noshadow'
```

```
{"value":"198.51.100.23","first_seen":1791668207,"last_seen":1791668230,
 "count":5,"tags":"tlp:green","ttl":0,"consensus":1}
```

`noshadow` matters. Without it the read is recorded as a search, in a `_shadow/` namespace beside the one you read. That is a feature — it answers "which indicators do my analysts keep looking up?" — and a nuisance when you are reading your own data in a loop.

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/r/_shadow/feeds/misp/ips?val=198.51.100.23&noshadow'
```

5.2.2 With statistics

`/rs` is `/r` plus the hourly breakdown:

```
{"value":"198.51.100.23", ..., "stats":{"1791666000":3,"1791669600":2}}
```

The keys are the Unix timestamp of the hour.

5.2.3 A whole namespace

```
curl -H 'Authorization: changeme' 'http://localhost:9999/r/feeds/misp/ips'
```

5.2.4 Just the size of one

```
curl -H 'Authorization: changeme' 'http://localhost:9999/r/feeds/misp/ips?count'
```

```
{"namespace":"feeds/misp/ips","values":1284}
```

Maintained as values are written and deleted, so it is a lookup rather than a walk — ask it as often as you like.

5.2.5 Many values at once

```
curl -H 'Authorization: changeme' -H 'Content-Type: application/json' \
  -X POST http://localhost:9999/rb -d '{"items":[
    {"namespace":"feeds/misp/ips","value":"198.51.100.23","noshadow":true},
    {"namespace":"feeds/misp/ips","value":"203.0.113.99","noshadow":true}
  ]}'
```

```
{"items":[
  {"value":"198.51.100.23","first_seen":1791668207,"last_seen":1791668230,
   "count":5,"tags":"tlp:green","ttl":0,"consensus":1},
  {"error":"Value not found","namespace":"feeds/misp/ips","value":"203.0.113.99"}]}
```

Results are positional: item n answers item n, and a miss is an error object in place of an attribute rather than a gap. `/rbs` is the same with statistics.

5.3 Deleting

```
curl -H 'Authorization: changeme' 'http://localhost:9999/d/feeds/misp/ips'
```

Deletes the **whole namespace**. There is no delete-one-value route: a sighting is a count over a window, and removing one observation from it is not a coherent operation. Expire values with a `ttl` instead.

5.4 What the status codes mean

Code	Meaning
`200`	Done. For a batch, possibly `partial` — read `items`.
`400`	The request is malformed, or a batch failed for mixed reasons.
`401`	No key.
`403`	The key may not do this, or you wrote to an internal namespace.
`404`	No such namespace, or no such value in it.
`421`	**Nowhere in reach stores that namespace.** See chapter 8.
`500`	Something broke; the message says what.
`502`	A mirror that holds this could not be reached.
`508`	A forwarded request ran out of hops; your galaxy has a cycle.

`421` and `502` only appear in a galaxy, and the distinction between them and `404` is deliberate: "no such namespace", "nowhere in reach stores it" and "the server that has it is down" are three different answers, and only one of them is worth retrying.

5.5 Namespaces beginning with _

Internal. `_all` is the consensus tally, `_shadow/*` is what has been searched for, `_config` is server state.

They are **readable** — `/r/_all?val=x` is how you ask how many namespaces have seen a value — and **not writable from outside**, by anything, at any permission level. A client that could write `_all` could forge consensus: one honest write plus five forged ones, and a value looks corroborated by six independent feeds. `_config` is not readable either.

5.6 The route list

```
curl http://localhost:9999/
```

Needs no key and lists every route with a line each. `GET /i` reports the version, and `GET /_api/openapi.yaml` is the whole API as an OpenAPI 3 document, which a test keeps honest.

6

Tags

A sighting on its own is *namespace, value, count, first seen, last seen*. That answers "how often, and when" and nothing else — it does not say what the value **is**, who saw it, or how it may be shared. Tags carry that.

6.1 The shape of a tag set

A comma-separated set, each entry either a bare label or `key:value`:

```
stix-type:ipv4-addr, tlp:amber, confidence:80, identity:Beta Cyber Intelligence
```

Comma is the separator because the interesting values contain spaces — an identity, a description — so a tag may contain **anything except a comma**, colons included. That is what a URL and an RFC 3339 timestamp need. Whitespace around entries is trimmed, and a key may repeat.

6.2 They merge

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/w/feeds/ips?val=198.51.100.23&tags=stix-type:ipv4-addr'
curl -H 'Authorization: changeme' \
  'http://localhost:9999/w/feeds/ips?val=198.51.100.23&tags=tlp:amber'
```

The value now carries both. Two feeds that each know something different about an address both contribute, and neither erases the other.

Which means **a write can never take a tag off**. That is the management interface's job:

```
curl -H 'Authorization: changeme' -H 'Content-Type: application/json' \
  -X POST http://localhost:9999/_management/api/tags \
  -d '{"namespace":"feeds/ips","value":"198.51.100.23","tags":"tlp:green"}'
```

That replaces the set outright. It is not a sighting: nothing is counted and no timestamp moves.

> In a galaxy the replacement is pushed to every mirror as it is made, and the answer says which took it. A tag change is not a sighting, so nothing else would ever carry it. Chapter 9.

6.3 The vocabulary the export reads

Anything you like is kept, and these are the keys the STIX export understands:

Tag	What it does
`stix-type:<type>`	The observable type, and so the pattern: `ipv4-addr`, `domain-name`, `url`, `email-addr`, `mutex`, `windows-registry-key`, or `file.<ALGORITHM>` for a hash.
`stix-id:indicator--<uuid>`	Use this indicator id rather than deriving one. Written by the STIX importer, so a value that arrived as STIX goes back out under the id its publisher gave it. See chapter 11.
`indicator-type:<value>`	Adds to `indicator_types`. Repeatable.
`tlp:<label>`	Marks the indicator and the sighting with the TLP marking.
`confidence:<0-100>`	STIX `confidence` on both objects.
`identity:<name>`	Who saw it. Becomes an `identity` object. Repeatable.
`name:`, `description:`	Indicator name and description.
`valid-until:`	Indicator `valid_until`, as RFC 3339 or Unix seconds.

The MISP importer also writes `misp-type:`, `misp-category:`, `misp-event:` and MISP's own tags as they were published.

6.4 Colours, and what a tag means

The **Tags** page gives every tag a colour and a description, and that is what makes a marking readable at a glance instead of being grey text among twenty others.

It decides **presentation only**. A value can carry a tag nothing here defines and that is not an error — a feed brings what it brings, and refusing tags to keep a table tidy would lose data.

6.4.1 Families

A name ending in `:` is a family and colours everything beneath it:

```
stix-type:          colours stix-type:ipv4-addr, stix-type:domain-name, ...
misp-galaxy:        colours every MISP galaxy tag
```

Half the vocabulary above is `key:value` with an open set of values, which could not be enumerated even in principle. An exact entry beats its family, and the longest family wins. It is also

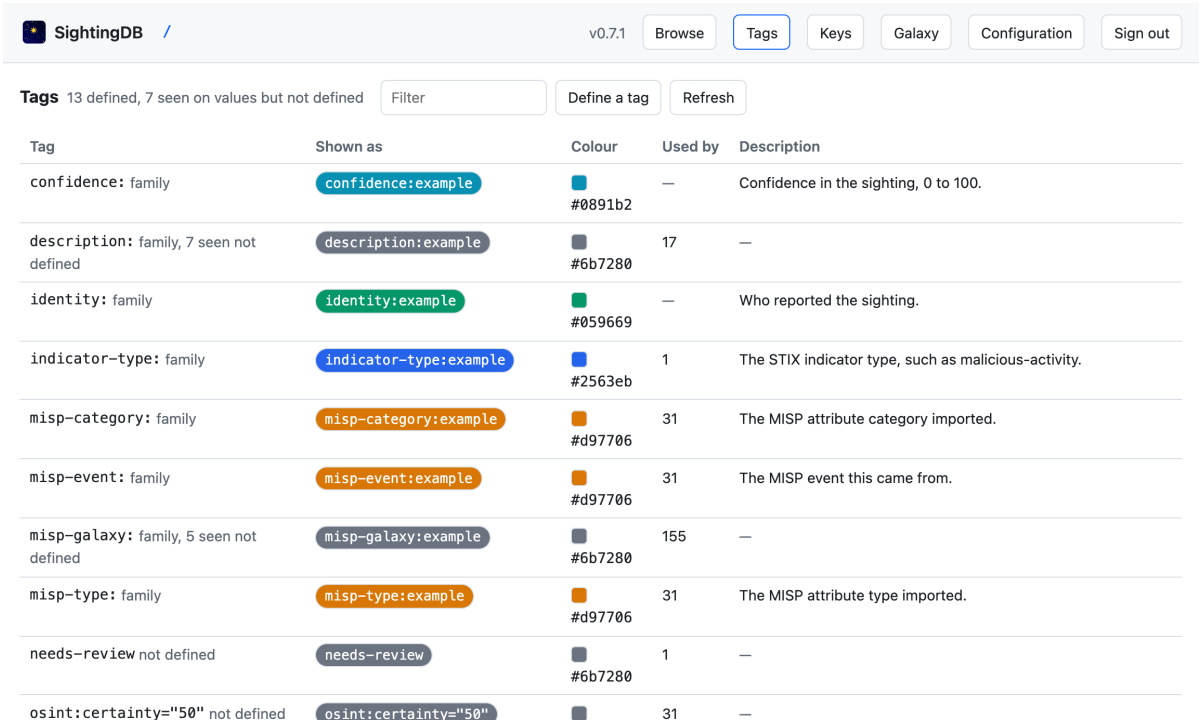


Figure 6.1: The tag vocabulary. Families, usage counts, and tags seen but not defined.

the shape of a MISP machine tag — `namespace:predicate` — so a whole MISP taxonomy can be coloured by its namespace.

6.4.2 What ships

A new installation starts with the five TLP 2.0 labels — `tlp:clear`, `tlp:green`, `tlp:amber`, `tlp:amber+strict`, `tlp:red` — plus `tlp:white`, which 2.0 renamed to CLEAR, because sources still send it. CIRCL's public OSINT feed tags its events `tlp:white` *and* `tlp:clear`, so a real import carries both; WHITE is given CLEAR's colour, because that is what it means. A colour per family of SightingDB's own vocabulary comes with them.

The colours are MISP's own, from its `tlp` taxonomy, which follows FIRST's, so a tag exported to MISP and back looks the same in both.

Seeded, not insisted upon. Once there is a `tags_file` the table is yours: any of them can be recoloured or removed, and nothing puts them back.

```
curl -H 'Authorization: changeme' -H 'Content-Type: application/json' \
  -X POST http://localhost:9999/_management/api/tags/vocabulary \
  -d '{"name":"needs-review","colour":"#AA33CC","description":"Ours, not from a feed."}'
```

```
curl -H 'Authorization: changeme' -X DELETE \
  'http://localhost:9999/_management/api/tags/vocabulary?tag=needs-review'
```

Removing a colour **does not remove the tag from the values**. This is a presentation setting, and deleting data from a colour picker would be a trap — so the tag stays in the table, listed as undefined, for as long as anything carries it.

Changing the table needs an `admin` key, or one with read *and* write across all namespaces. A key that can already write any tag onto any value is not meaningfully restrained by being unable to say what colour it is shown in — but a key scoped to a subtree cannot, since the vocabulary is server-wide and changing it would reach past that scope.

6.4.3 Tags found on values

The page lists tags nothing has defined, so adopting one is a click rather than a discovery. Those are **grouped where a key is being used as a free-text field**: MISP turns an attribute's comment into `description:<text>`, and real comments are paragraphs — a single imported CIRCL event produced seven rows of prose. A key with more than three distinct values is shown once, as its family, saying how many it stands for. Three or fewer keep their own rows, because that is a vocabulary rather than a free-text field, and giving every `tlp:` label one colour would defeat the point of a marking.

6.4.4 Typing one

The tag box completes what you are typing, offering tags already in use. It completes the entry after the last comma rather than the whole field, because the point is to type a tag the same way twice. A family is offered with its colon kept, leaving the cursor ready for the value.

6.5 Where tags show up

In `/r` and `/rs` responses, in the DNS TXT answer, in the STIX export, and everywhere a value appears in the interface.

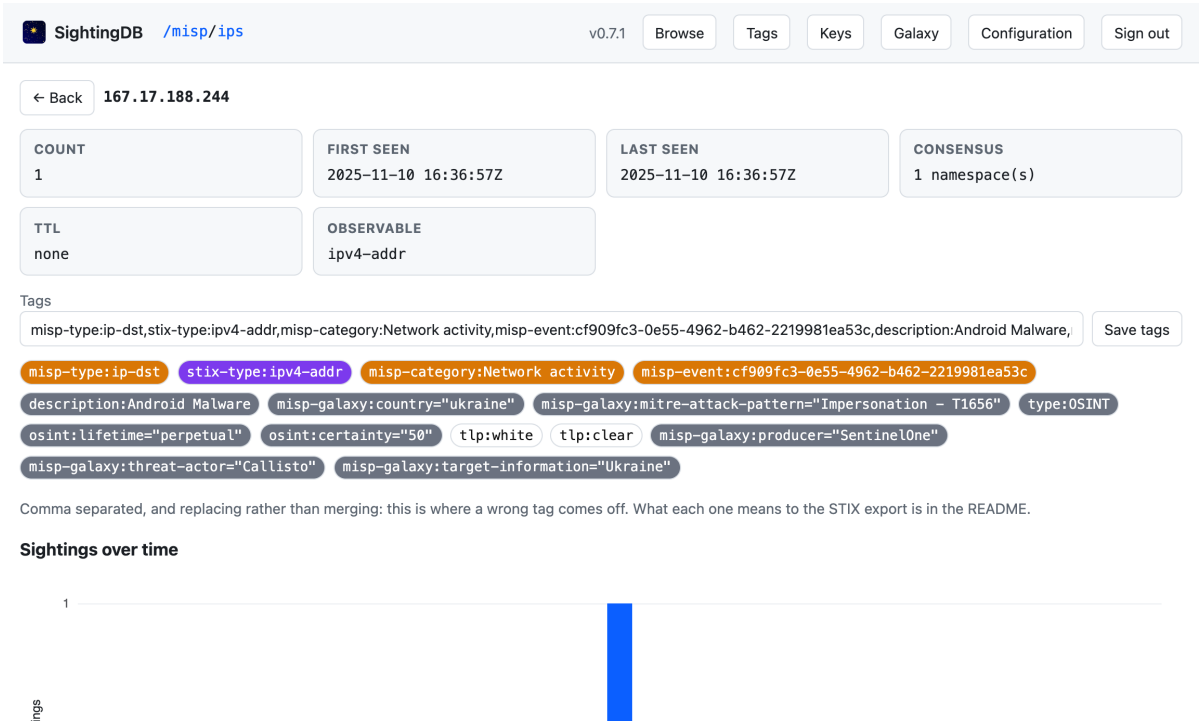


Figure 6.2: A value, its history and its tags.

7

The Python client

```
pip install sightingdb-client
```

Python 3.10 or newer. The source is at `github.com/stricaud/sightingdb-client`.

7.1 Connecting

```
from sightingdb import SightingDB

db = SightingDB(url="https://sightingdb.example:9999", apikey="changeme")
```

Or from the environment, which is what you want in anything scheduled:

```
export SIGHTINGDB_URL=https://sightingdb.example:9999
export SIGHTINGDB_APIKEY=changeme
```

```
db = SightingDB()
```

Or from `~/.config/sightingdb/client.toml`. `client.toml.example` in the repository is the template. For a self-signed certificate, `verify=False`.

7.2 Writing

```
db.write("feeds/misp/ips", "198.51.100.23")
```

Returns the new count. Timestamps and TTLs are `datetime` and `timedelta` rather than seconds, because that is what your code already has:

```
from datetime import datetime, timedelta, timezone

db.write("feeds/misp/ips", "203.0.113.7",
         timestamp=datetime(2026, 3, 1, tzinfo=timezone.utc),
         ttl=timedelta(days=30))
```

7.3 Writing a lot

One call per sighting is one HTTP request per sighting. For ingest, batch:

```
from sightingdb import Sighting

result = db.write_many([
    Sighting("feeds/misp/ips", "198.51.100.23", ttl=timedelta(days=7)),
    ("feeds/misp/ips", "203.0.113.7"),
    {"namespace": "feeds/misp/domains", "value": "login.phish.example"},
])

print(result.written, "written")
for failure in result.errors:
    print(failure.namespace, failure.value, failure.error)
```

Three shapes are accepted because three shapes are what callers have: a `Sighting`, a tuple, or the dict you already decoded from somewhere.

`strict=True` — the default — raises if anything was refused. `strict=False` returns the result and lets you decide, which is usually what a feed importer wants: one bad value out of fifty thousand should not stop the import.

For a stream that never ends, a batch flushes itself:

```
with db.batch(chunk_size=5000) as batch:
    for namespace, value in every_line_of_the_feed():
        batch.add(namespace, value)
```

7.4 Reading

```
seen = db.read("feeds/misp/ips", "198.51.100.23", shadow=False)

print(seen.count)
print(seen.first_seen_at, seen.last_seen_at)   # datetimes
print(seen.tags)
print(seen.consensus)
```

`shadow=False` is `noshadow` from chapter 5: it stops the read being recorded as a search. **Pass it whenever you are reading your own data**, and leave it alone when the read represents an analyst actually asking.

Hourly statistics on request:

```
seen = db.read("feeds/misp/ips", "198.51.100.23", stats=True, shadow=False)
for hour, count in seen.stats_by_hour.items():
    print(hour, count)
```

Many at once, positionally:

```
results = db.read_many([
    Sighting("feeds/misp/ips", "198.51.100.23", noshadow=True),
    Sighting("feeds/misp/ips", "203.0.113.99", noshadow=True),
])

for result in results:
    if result.found:
        print(result.value, result.count)
    else:
        print(result.value, "not seen")
```

And the question you will ask most:

```
if not db.exists("feeds/misp/ips", address, shadow=False):
    alert("first time we have ever seen this")
```

7.5 The rest

```
db.list_values("feeds/misp/ips", shadow=False)   # every value, as Attributes
db.delete("feeds/misp/ips", missing_ok=True)     # the whole namespace
db.info()                                         # version, what it stores
db.ping()                                         # is it there
db.close()
```

`SightingDB` is a context manager, so `with SightingDB() as db:` closes the connection for you.

7.6 Asynchronously

`AsyncSightingDB` has the same API with `await`:

```
from sightingdb import AsyncSightingDB

async with AsyncSightingDB(url=..., apikey=...) as db:
    await db.write("feeds/misp/ips", "198.51.100.23")
    seen = await db.read("feeds/misp/ips", "198.51.100.23", shadow=False)
```

7.7 Talking to several servers

A client can point at a load balancer — chapter 8 — and nothing changes. It can also fan out itself, which is useful when you would rather not run one:

```
db = SightingDB(
    urls=["https://node-a.example:9999", "https://node-b.example:9999"],
```

```
    apikey="changeme",
)
```

Writes then go to **every** node and reads to **one**. Two things make that correct rather than merely parallel, and both are easy to get wrong.

A fanned-out write is attributed to its origin. SightingDB counts sightings per server so that merging two copies adds separate contributions rather than replaying increments. One write sent to two nodes would otherwise be counted by each under its own name, and when those nodes sync, three writes would become six. So the client names itself in a header and every node counts the write under that one name.

A value always reads from the same node. Nodes drift apart transiently while they catch up with each other, so two consecutive reads served by different nodes could show a count going *down*. The client picks the node by hashing the value, which also moves the fewest values when the node list changes.

```
db.fans_out      # True when there is more than one
db.urls          # what it is pointing at
```

With one URL none of this happens: no extra headers, no hashing. Pointing at one server behaves exactly as it did before any of this existed.

7.8 A worked example

Enrich a stream of addresses, recording each as seen and reporting the new ones:

```
from datetime import timedelta
from sightingdb import SightingDB, Sighting

with SightingDB() as db:
    addresses = [line.strip() for line in open("today.txt")]

    # One round trip to find out which are new...
    before = {
        result.value: result.found
        for result in db.read_many(
            [Sighting("myorg/proxy/ips", a, noshadow=True) for a in addresses]
        )
    }

    # ...and one to record them all.
    db.write_many(
        [Sighting("myorg/proxy/ips", a, ttl=timedelta(days=90)) for a in addresses],
        strict=False,
    )

    for address in addresses:
```

```
        if not before.get(address):
            print("first sighting:", address)
```

Two requests for the whole file, rather than two per address.

8

Galaxies

One server is a *node*. Several servers that know about each other are a *galaxy*. This chapter is what the pieces are; the next is running one.

8.1 Why more than one

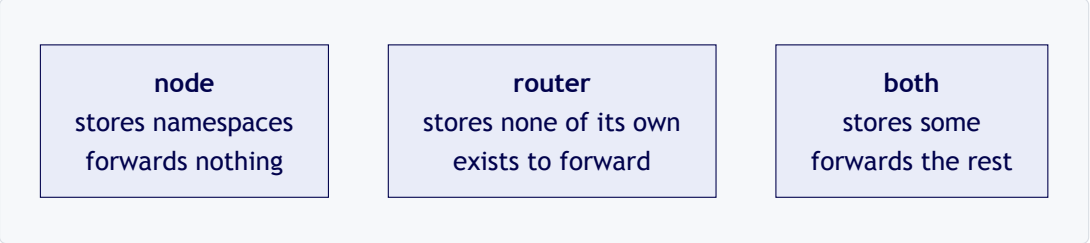
Two reasons, and they want different shapes.

More clients than one server can answer. Put a server in front that stores nothing and forwards everything to mirrors behind it. Reads spread across the mirrors; writes go to all of them.

More data than one server should hold. Give each node a slice of the namespace tree. A node that holds only `feeds/misp` needs the disk for `feeds/misp` and nothing else.

Both are the same mechanism.

8.2 The three roles

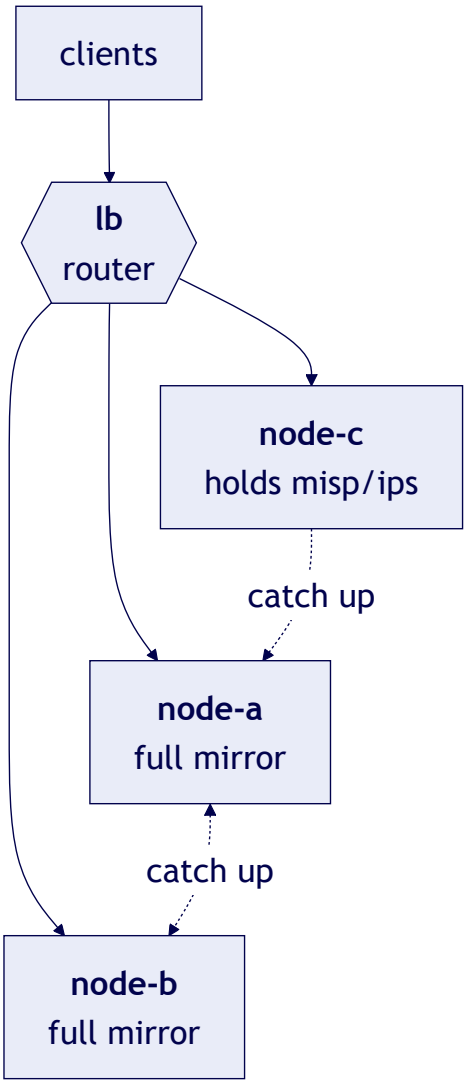


They describe a *configuration*, not a type. Any server can be any of them, and the only thing that decides which is `[storage] namespaces`:

`namespaces`	Role
absent	A node that stores everything — a full mirror.
`["feeds", "myorg"]`	A node that stores those subtrees — a partial mirror.
`[]`	A router. Stores nothing of its own.

Add `[galaxy] peers` and a node becomes "both": it keeps what it holds and forwards what it does not.

8.3 What a galaxy looks like

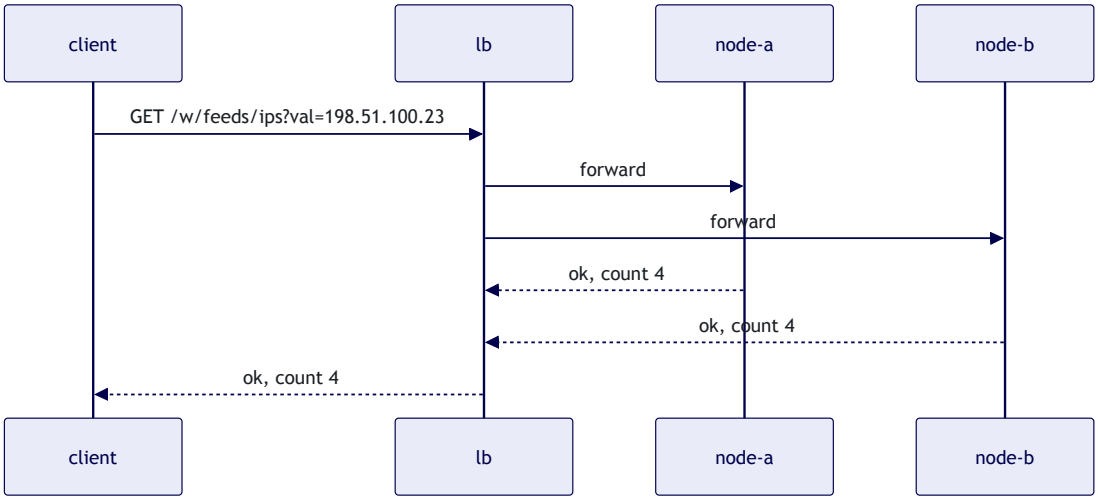


That is the sample in `doc/docker`. The load balancer is the entry point; clients talk only to it. Each node declares what it holds, and the load balancer is configured with the same declaration so it knows where a request can be served.

8.4 Forwarding

The rule is short:

- **A write goes to every live mirror** of its namespace.
- **A read goes to one**, chosen by hashing the value.



Writes must reach every mirror or the mirrors diverge. Reads must reach exactly one, and always the *same* one for a given value, for a reason worth stating: mirrors drift apart transiently while they catch up with each other, so two consecutive reads served by different mirrors could show a count going **down**.

The choice is made by hashing the value against each mirror's address and taking the highest — *rendezvous hashing*. Compared with picking a mirror by `hash(value) % n`, it moves the fewest values when a mirror is added or removed: mod-n reshuffles about 89% of values where 11% is the minimum. That was measured rather than assumed; `doc/sharding-experiment.py` reproduces it.

8.5 Counting, without inflating

Here is the problem that shapes everything else.

A write arrives at the load balancer and is forwarded to two mirrors. Each records a sighting. Later the mirrors sync with each other. If each counted the write as *its own*, the merge would add the two together and three writes would become six — and six again at the next merge.

So **a sighting is counted per server**:

```
{"counts": {"lb": 3}, "first_seen": ..., "last_seen": ...}
```

Both mirrors record `{"lb": 3}`, because the write is attributed to the entry point the client actually reached and not to whichever mirror stored it. Merging two copies takes the greater value for each server, so two mirrors that already agree change nothing. The total you read is the sum.

That single decision is what makes every other piece safe: merges can be retried, they can arrive in any order, and a mirror can be synced twice with no effect.

> This is why `node_id` must be unique. Two servers sharing a name each take the other's contributions for their own, and the merge meant to reconcile them discards one side instead.

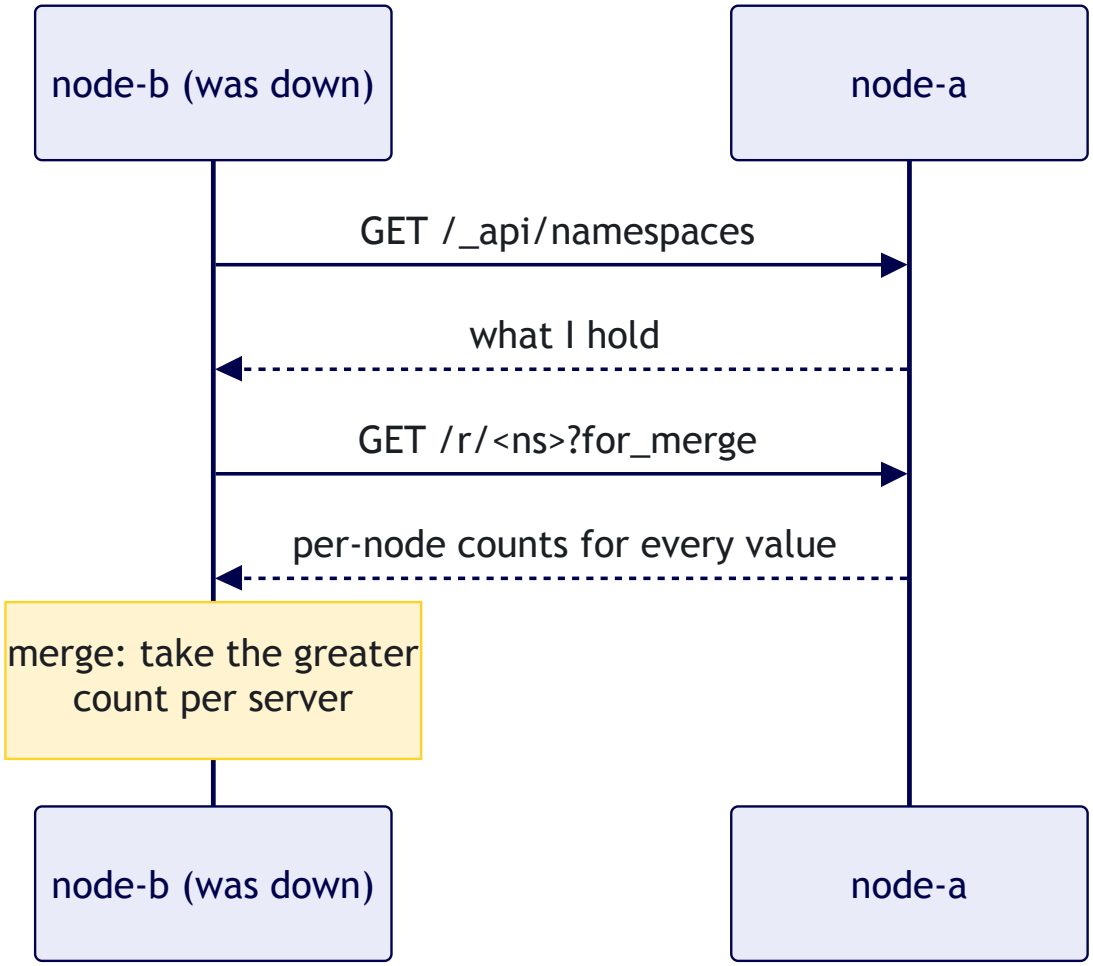
8.6 Consensus across a galaxy

`consensus` — how many namespaces have seen a value — cannot be answered by any single node, because no node has all the namespaces. The router keeps the tally itself: when a mirror says a sighting was *new there*, the router counts it.

That number only ever rises on its own, because values expire and namespaces are deleted on the nodes and neither reaches the router. So a router surveys its galaxy on a timer and puts the tally right. It is a repair, not a steady cost: the incremental tally answers reads in between.

8.7 Catching up

A node that was down missed whatever was written while it was away. On a timer it asks each peer what it holds, compares, and pulls what it is missing.

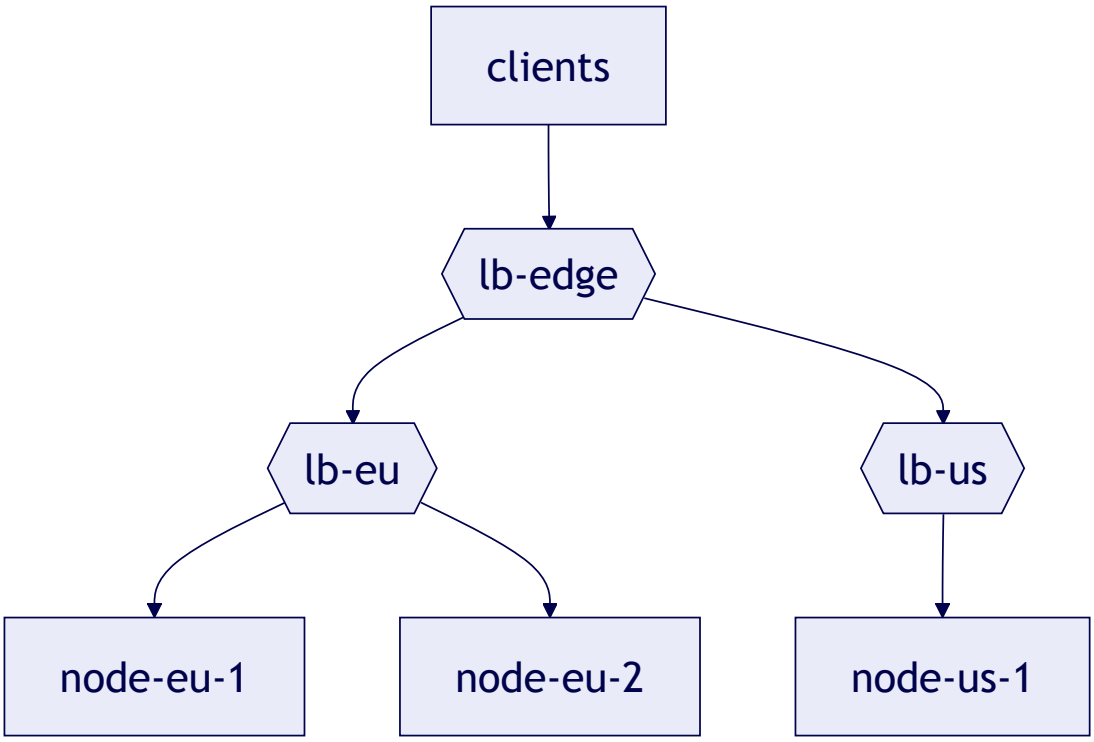


It only ever pulls namespaces it stores itself, so a partial mirror stays partial.

While a node is catching up it says so, and **the router sends reads elsewhere** until it is done — which is what makes a backfill invisible rather than a window of wrong answers. Writes keep going to it throughout: they land directly, and the catch-up fills in the history behind them.

8.8 Cascades

A peer may itself be a router, so a galaxy can be a tree:



A forwarded request carries a hop count, spent at every hop and refused at zero. That is what stops a miswired cycle — and a cycle matters more here than usual, because a request going round one would inflate every count it carried. A request that runs out of hops answers `508 Loop Detected`.

A cascade also **preserves the entry point**: a write forwarded twice is still counted once, for the server the client actually reached.

8.9 Keys are the boundary

Every peer entry carries the key *this* server authenticates to that peer with:

```
peers = [
  { url = "https://node-a:9999", key = "lb-full" },
  { url = "https://node-c:9999", key = "lb-ips", namespaces = ["misp/ips"] },
]
```

Those keys live in the **peers'** ACLs, and it is the peer that enforces them. Give the load balancer `rw:misp/ips` on node-c and it cannot write anywhere else there — however it is configured, and however it is compromised. That is not a policy the load balancer applies to itself; it is a capability it does not hold.

Give a peer key the narrowest grant that does the job, and never `admin`: nothing in forwarding needs it.

> One consequence worth knowing: the galaxy *graph* asks each peer about its own peers, and that needs `admin` there. With narrow keys the graph draws this server's peers from its own configuration instead, labelled "as configured here". That is the right trade, not a bug.

8.10 When nothing holds it

Answer	Means
`404`	The namespace exists somewhere in reach, and the value is not in it.
`421`	**No server in reach stores that namespace at all.**
`502`	A mirror that holds it could not be reached.
`508`	The request ran out of hops.

Three different answers, and only `502` is worth retrying. Collapsing them into `404` — which is what an earlier version did — makes "the server that has it is down" indistinguishable from "there is no such thing".

9

Running a galaxy

Chapter 8 was what the pieces are. This is building one, changing it, and taking parts of it out of service.

9.1 Build one in four containers

The fastest way to see all of it:

```
docker compose -f doc/docker/docker-compose.yml up --build
```

A load balancer on 9999 and three nodes on 9991–9993, two full mirrors and one holding a single namespace, loaded with real indicators from CIRCL's public OSINT feed. `doc/docker/README.md` lists things to try. The rest of this chapter is what it is doing.

9.2 Adding a node

A load balancer takes a new node over HTTP, and starts using it at once — no restart, which matters because restarting a router is a gap in service for everything behind it.

First, on the new node, create the key the load balancer will use. It is the node's own ACL that decides what that key opens, so make it the narrowest thing that works:

```
curl -H 'Authorization: <node admin key>' \
  -X POST https://new-node.example:9999/_management/api/keys \
  -H 'Content-Type: application/json' \
  -d '{"key": "lb-to-new-node", "admin": false,
       "read": [""], "write": [""]}'
```

`read` and `write` are lists of namespace prefixes and `""` means all of them, so that is read and write everywhere — what a full mirror needs. For a node that only holds `feeds`, name it and the key cannot reach past it:

```
  -d '{"key": "lb-to-feeds", "admin": false,
       "read": ["feeds"], "write": ["feeds"]}'
```

Then, on the load balancer, add the peer:

```
curl -H 'Authorization: <lb admin key>' \
  -X POST https://lb.example:9999/_management/api/galaxy/peers \
  -H 'Content-Type: application/json' \
  -d '{"url": "https://new-node.example:9999", "key": "lb-to-new-node"}'
```

Leave `namespaces` out for a full mirror; name them for a partial one. It must agree with the node's own `[storage] namespaces`.

The answer is the whole peer list, so nothing else has to be fetched. The same thing is a form on the **Galaxy** page.

This needs a `peers_file` in `[galaxy]`. Without one the galaxy is read-only over the API, and the page says so rather than letting a save fail.

9.2.1 The node does not have to be up

A mirror that is down must still be addable, or a galaxy could not be rebuilt after whatever took it down. It is shown offline until it answers.

9.2.2 Give the new node peers of its own

This is the part that is easy to miss, and it is measured rather than theoretical.

A new node starts **empty**. From the moment it is added, writes fan out to it — but reads are spread by hashing, so a share of reads goes to a node that does not have the history yet. With twenty values on an existing node and an empty node added beside it:

```
reads through the load balancer after adding the empty node: 13/20 now MISS
```

Thirteen values that are in the galaxy, reported absent. And it does not heal: the node has no way to know what it is missing.

The fix is to give the new node a `[galaxy]` section naming at least one existing node, and a non-zero `sync_interval`:

```
[galaxy]
sync_interval = 300
peers = [
  { url = "https://node-a.example:9999", key = "new-node-to-node-a" },
]
```

It then asks that node what it holds, compares, and pulls what it is missing. While it does, it reports `catching_up` and the load balancer sends reads elsewhere. With that in place, the same test:

```
right after it starts    node-d: holds=['feeds/ips']  misses=0/20
```

The key for *that* direction is read-only — pulling is a read, and a node catching up has no business writing on the node it pulls from:

```
  -d '{"key": "new-node-to-node-a", "admin": false,
       "read": [""], "write": []}'
```

A node with no `[galaxy]` at all never reports catching up, because it has nothing to catch up from. That is a valid configuration — a node that only ever holds what arrives from now on — as long as it is the one you meant.

9.3 Changing one

Rotating a key, or changing what a node holds, is `PUT`:

```
curl -H 'Authorization: <lb admin key>' \
  -X PUT https://lb.example:9999/_management/api/galaxy/peers \
  -H 'Content-Type: application/json' \
  -d '{"url": "https://new-node.example:9999", "key": "a-rotated-key"}'
```

Adding and changing are separate verbs on purpose. An upsert would mean that typing an address that already exists silently replaces its key, and that key is the one thing bounding what the load balancer may do there. `POST` on an address already present answers `409`; `PUT` on one that is absent answers `404`.

The key is required even when only `namespaces` changes, because it is never read back: a peer key is a credential this server holds, and a topology view is not a reason to hand it out.

9.4 Taking one out of service

```
curl -H 'Authorization: <lb admin key>' \
  -X POST https://lb.example:9999/_management/api/galaxy/peers/enabled \
  -H 'Content-Type: application/json' \
  -d '{"url": "https://node-b.example:9999", "enabled": false}'
```

The peer is **kept** — address, key, what it holds — and this server sends it nothing at all: no forwarded request, no catch-up, no gossip, not even a health probe. For a node going down for maintenance, or one you want to stop sending data to without having to find its key again to put it back. `Disable` and `Enable` on the Galaxy page are the same call.

It survives a restart, because it is written to the `peers_file`:

```
[[peers]]
url = "https://node-b.example:9999"
key = "lb-full"
enabled = false
```

A namespace only that peer held is out of reach while it is off, and reads of it answer `421`. That is what taking a node out of service means rather than a side effect of it — check the `Holds` column before you disable something.

Rotating a key leaves a peer disabled if it was. Two different decisions.

9.5 Removing one

```
curl -H 'Authorization: <lb admin key>' -X DELETE \
  'https://lb.example:9999/_management/api/galaxy/peers?url=https%3A%2F%2Fnode-b.example%3A9999'
```

Nothing stored on the node is deleted; this server stops using it. Its health is forgotten with it, so re-adding the same address starts unprobed rather than inheriting a stale "offline".

9.6 What you cannot change over the API

A peer written into `[galaxy] peers` in the configuration file:

```
{"message":"https://node-a.example:9999 is declared in the configuration file,
 so removing it here would not last: it would come back at the next restart.
 Remove it from [galaxy] peers instead."}
```

That file is yours and the program does not rewrite it, so a change made here would be undone by the next restart. Either edit it, or move the peer out of it so the API owns it. `sightingdb --setup` writes a live `[galaxy]` section with a `peers_file` and no peers in it, which is a valid galaxy and the easier starting point.

9.7 Watching it

```
curl -H 'Authorization: changeme' \
  https://lb.example:9999/_management/api/galaxy/peers
```

```
{"peers":[
  {"url":"https://node-a:9999","namespaces":null,"fixed":false,"enabled":true,
   "health":{"online":true,"probed":true,"last_seen":1791678296,
             "latency_ms":1,"version":"0.7.1","error":null,
             "failures":0,"catching_up":false}}],
 "editable":true}
```

`probed: false` with `online: false` means "not asked yet", which is not the same as down. `last_seen` is when this server last got a health reply from that peer — nothing to do with a value's own `last_seen`, which is why the interface calls the column **Last answered**.

The graph colours a full mirror, a partial mirror and a router differently, and red for a server that is not answering. `GET /_management/api/galaxy` walks the peers, so one request describes a whole cascade.

9.8 Keys across a galaxy

Each server has its own ACL, and the **Keys** page shows the ACL of the server you are connected to and nothing else. The keys a *node* accepts live on that node.

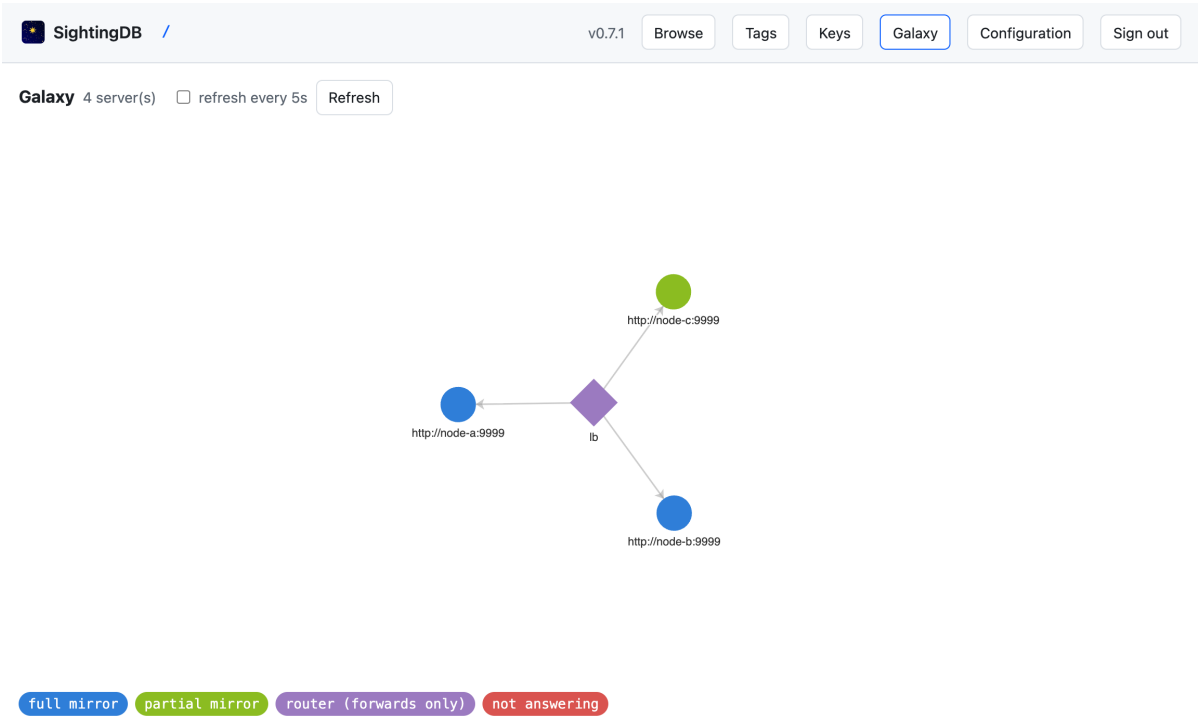


Figure 9.1: The galaxy, and the peers under it.

Two optional mechanisms connect them:

- `gossip_interval` pushes this server's keys to its peers, so a peer that was down for a key change picks it up. The push goes through the peer's own management interface, so the peer's ACL decides — this server can only administer a peer it holds an `admin` key for. The offer is *additive*: a revocation made while a peer was down does not reach it this way.
- `acl_authority` and `acl_replaceable` are the answer to that. Set `acl_authority` on the server that owns the galaxy's keys and `acl_replaceable` on each server that accepts them, and the owner's offer becomes the whole list, to be held exactly — which *does* remove. Both ends must agree, so a server with `acl_replaceable` off keeps its own keys and nothing is lost by not opting in. Setting both on one server is refused at startup: they would take turns overwriting each other.

`GET /_management/api/keys/drift` shows where a server's keys and its peers' disagree. It needs `admin` on the peer key, which a deliberately narrow peer key does not have — in which case it says so, which is a correct answer rather than a failure.

9.9 A checklist

Before a node joins:

- It has its **own** `node_id`. Nothing else in this chapter works if two servers share one.
- Its `[storage] namespaces` matches what the load balancer declares for it.
- It has a key for the load balancer, scoped to what that load balancer should reach.
- It has `[galaxy] peers` and a `sync_interval` of its own, so it backfills.
- The load balancer has a `peers_file`, so the addition survives a restart.

10

The management interface

`https://your-server:9999/_management`, with a key holding `admin`. On a fresh install that is `changeme`, and changing it is the first thing to do.

It is one page — markup and script in a single file compiled into the binary — so there is nothing to deploy and nothing to keep in step with the server.

10.1 Browsing

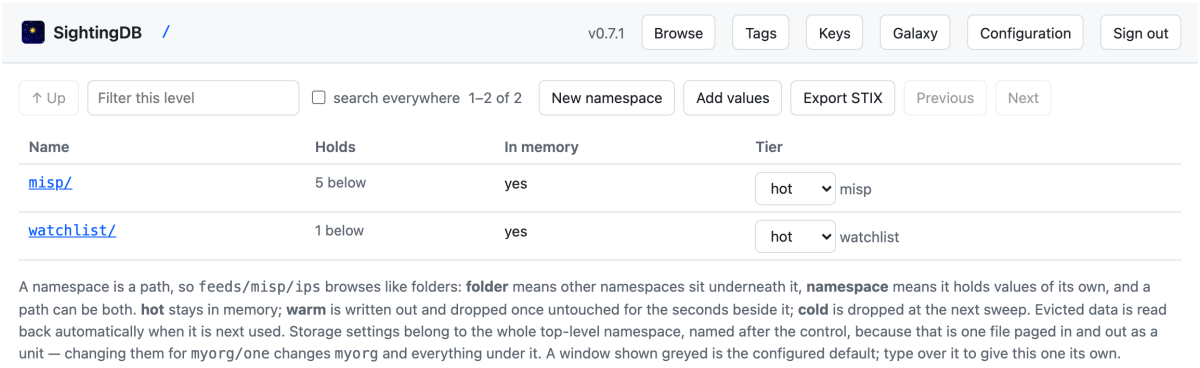


Figure 10.1: A namespace is a path, so it browses like folders.

folder means other namespaces sit underneath it, **namespace** means it holds values of its own, and a path can be both. The storage column is from chapter 4: `hot` stays in memory, `warm` is written out after the seconds beside it, `cold` goes at the next sweep. Those settings belong to the whole top-level namespace, because that is one file paged in and out as a unit — which the page says out loud when you change one.

10.2 Values

The **Observable** column is what the STIX export will call each value, sitting next to the tags that decide it — on purpose, because the column that tells you a value has no observable type is next to the one you fix it in.

Tags are editable in place, and the box completes what you type from tags already in use.

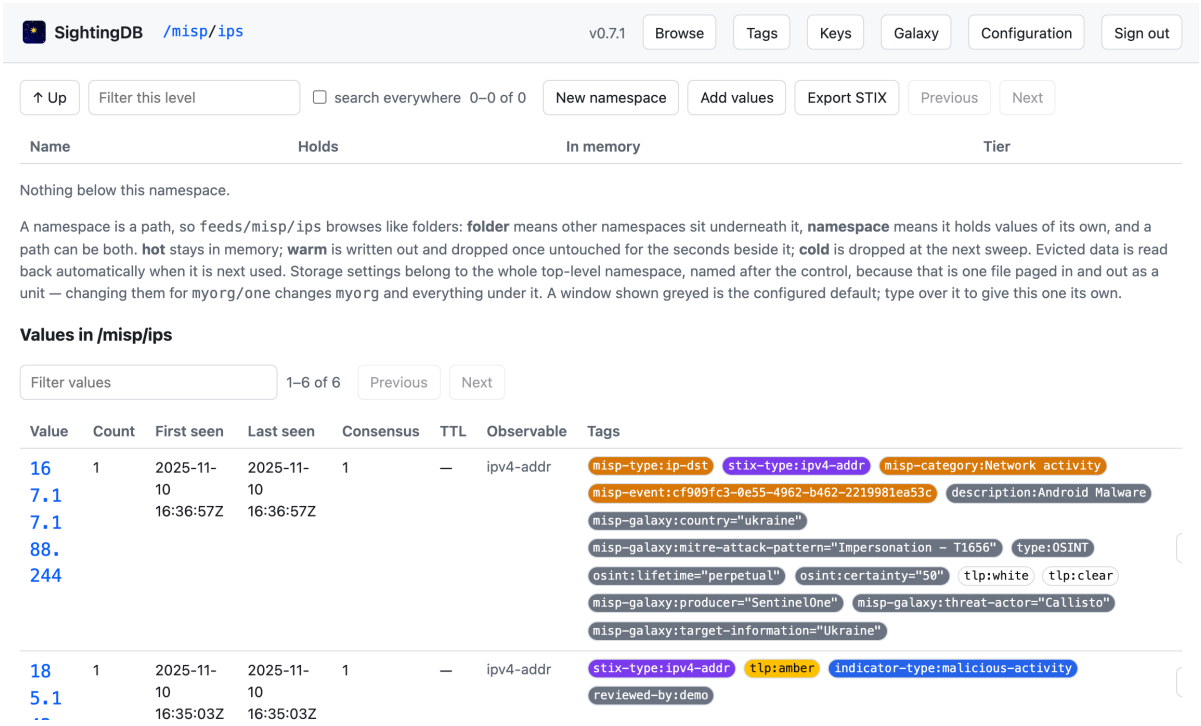


Figure 10.2: The values in a namespace, with what the export will make of them.

10.3 One value

The histogram is the hourly statistics from `/rs`. **Consensus** is how many namespaces have seen this value at all, and the list below says which — the answer to "is this just my proxy, or is it in three feeds as well?".

10.4 Tags

Chapter 6 is this page in full.

10.5 Galaxy

The graph is drawn from one request that walks the whole cascade. Below it, the peers table is where a node is added, re-keyed, disabled or removed — chapter 9.

Note what `Last answered` means: when this server last got a health reply from that peer. It is about the server, not about any value.

10.6 Activity

How busy each top-level namespace has been: every read and every accepted write, counted against the first segment of the namespace, so `feeds/misp/ips` counts towards `feeds`. It answers the questions the data itself cannot — is anyone still querying this feed, did the ingest stop on Tuesday, which customer is doing all the reading.

Each namespace is a row of cells, one per day or per hour, the way a status page draws uptime: **reads** in blue, **writes** in orange, darker for more. The shades are on one scale across every

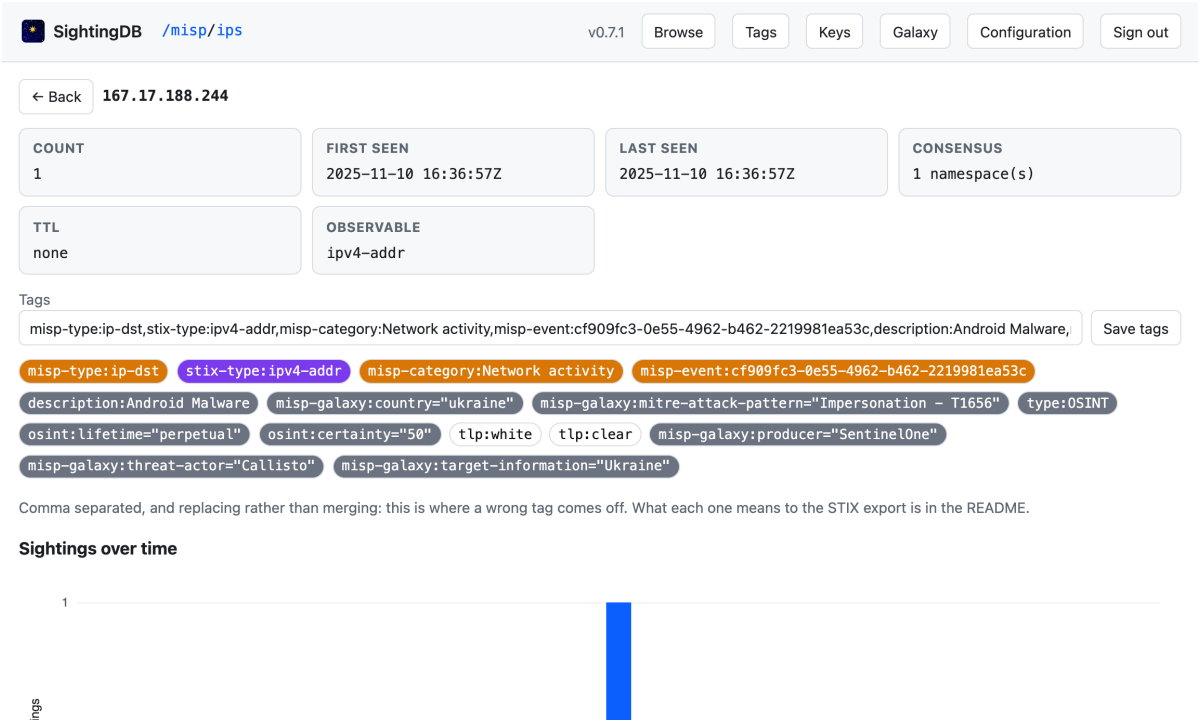


Figure 10.3: One value: its history, its tags, and where else it has been seen.

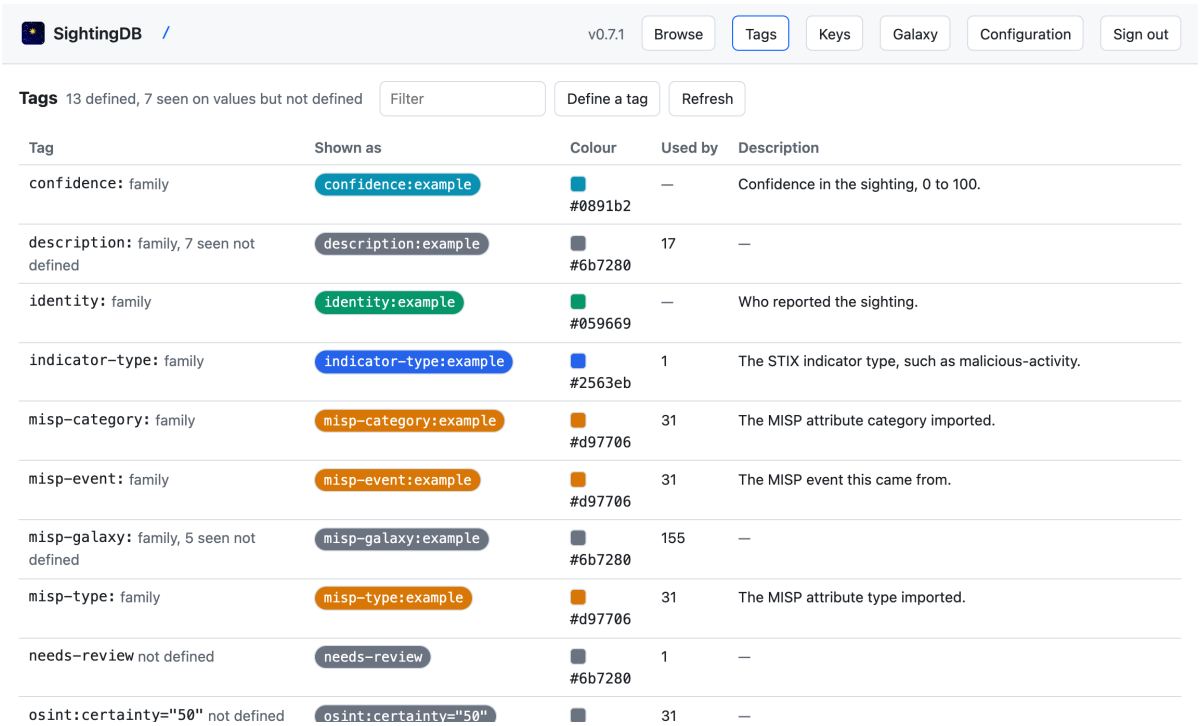


Figure 10.4: The vocabulary: colours, what each tag means, and what is in use.

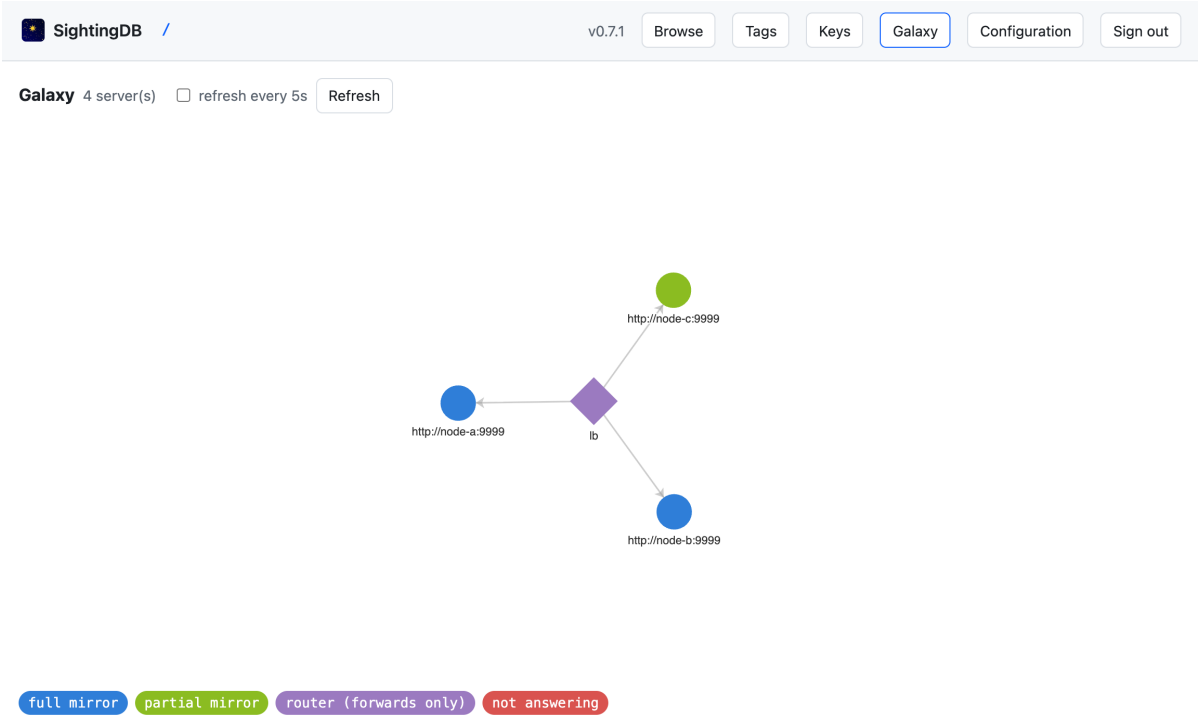


Figure 10.5: The galaxy, and the peers under it.

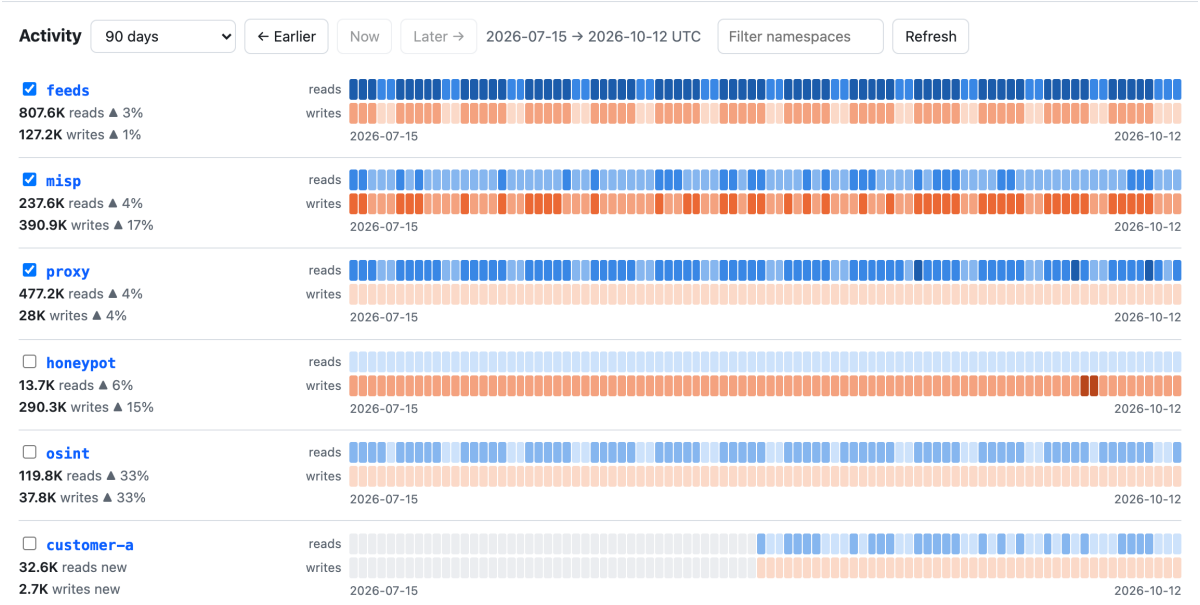


Figure 10.6: Ninety days, one row per namespace. The honeypot's scanning campaign and the customer that arrived six weeks ago are both visible without reading a number.

namespace on the page, so two cells of the same shade saw about the same traffic, whichever row they are in. A grey cell is a bucket in which nothing happened at all.

Beside each name are the totals for the window and how they compare with the window of the same length just before it — ▲ 17% against the previous ninety days, here. When that earlier window reaches back past what is kept, the comparison is left out rather than computed against half a window.

Browsing. The window is 24 or 48 hours, a week by the hour, or 30 days, 90 days or a year by the day. **Earlier** and **Later** move a whole window at a time, so nothing is shown twice; **Now** comes back. Hovering a cell gives its exact counts. **Clicking a day opens its 24 hours**, which is the way into a spike:

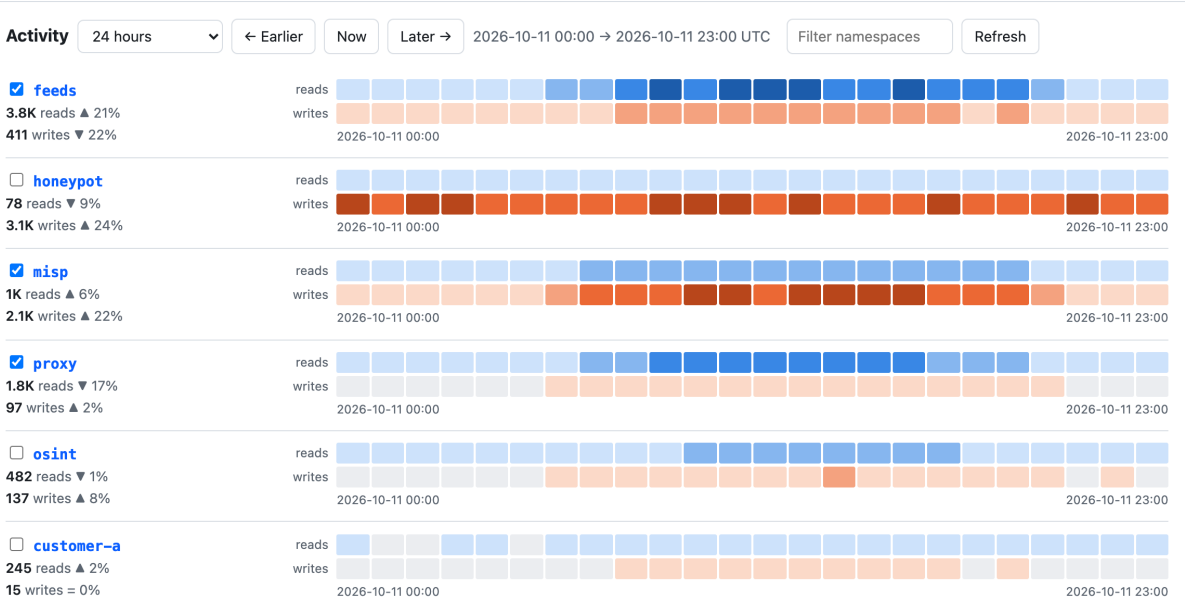


Figure 10.7: The day before, by the hour. Working hours show on every namespace but the honeypot, which is written to around the clock.

A year is drawn as a calendar instead, one column per week and one row per weekday, because 365 slivers in a row are unreadable. Gaps and weekly rhythms are what it is for:

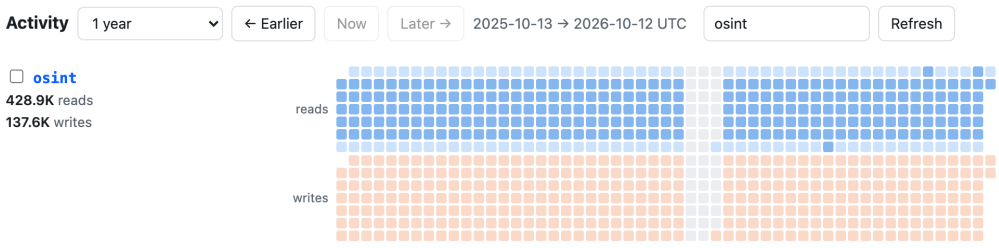


Figure 10.8: A year of one namespace. Twenty days when the feed delivered nothing, and nobody read it either.

Comparing. Tick namespaces to draw them on one chart below — the busiest three are ticked to begin with, and up to eight can be. It shows reads or writes, never both: they differ by orders of magnitude and would not share an axis.

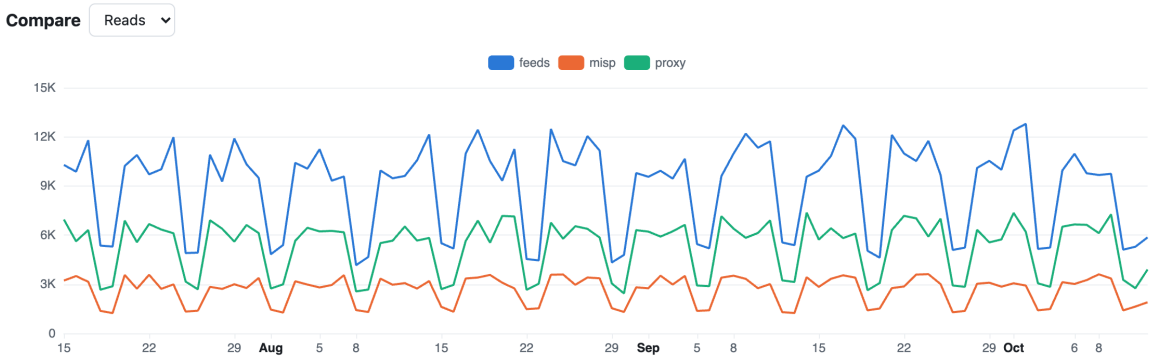


Figure 10.9: Three namespaces' reads on one chart.

What is counted is what clients asked for. Every route counts — `/r`, `/rb`, `/w`, `/wb`, DNS, the ZMQ ingest, and values added from this interface. A read of a value that is not there still counts; a write that was refused does not. The database's own bookkeeping — `_all`, `_shadow/*`, `_config/*` — is not counted, and neither are galaxy merges: a value one node forwards to another was a write on the first node, not a second write on the second.

Hours are kept for 31 days and days for 400, all in UTC. The counts are saved with snapshots as `activity.json` in `dbdir`, so they survive a restart. A namespace's row is shown only to a key that can read all of it: a total for `feeds` says something about everything under it.

Counting costs a brief lock on every read and write. If that matters more than the picture, `activity = false` in `[daemon]` turns it into a single flag check; the page then says counting is off, and still shows whatever was counted before.

10.7 Keys

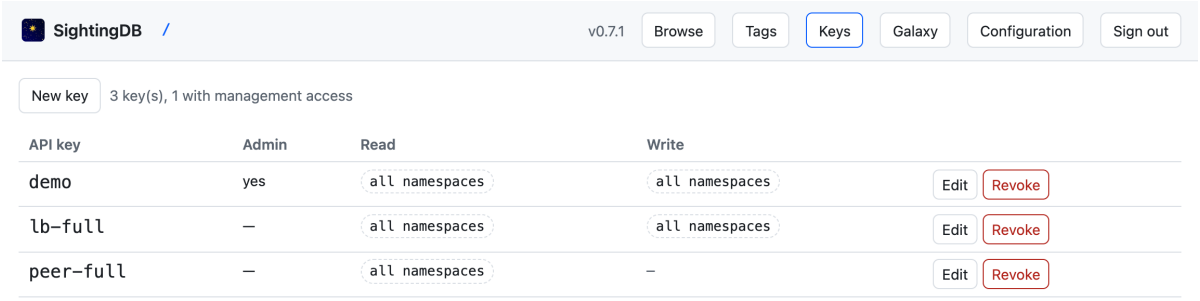


Figure 10.10: What each key may reach.

Create, scope and revoke. The guard rails refuse any change that would leave no admin key at all, by demotion or by deletion — there is no way to lock yourself out from this page.

Keys are per server. This page shows the ACL of the server you are connected to; the keys a node accepts live on that node. See chapter 9 for the two mechanisms that connect them.

10.8 Configuration

Read-only, and says so: it is what the server read at startup. Everything on it comes from `sightingdb.toml`, which this program does not rewrite.

The exceptions are the four machine-owned files from chapter 4 — keys, tiers, tag colours and peers — each edited on its own page.

It also shows the TLS certificate's expiry, and says **about to expire** when there are fewer than 30 days left. A certificate that quietly lapses takes a galaxy with it, since peers stop being able to authenticate to each other.

10.9 Exporting

Export STIX on a namespace asks two things before it runs: whether to include the sub-namespaces beneath it, and what to do with values whose observable type could not be worked out — leave them out, or export them flagged as untyped. Chapter 11.

10.10 Rejections

Every write path records what it turned away, newest first, including the ZMQ ingest which has no caller of its own to tell. It is the answer to "the feed says it sent fifty thousand and I have forty-nine".

It is bounded and in memory: a record kept to make something visible, not a fact about the data.

10.11 A note on caching

The page is served with `Cache-Control: no-cache`, so a browser revalidates it every time. It is the whole application in one file, and a cached copy means a browser running the previous version's JavaScript against this version's API — which looks like the server being wrong rather than the page being old.

If you ever see an interface that disagrees with the API, reload it once.

11

STIX and MISP

Chapter 1 made the claim that SightingDB stores exactly what a STIX sighting is. This is the chapter where that pays.

11.1 Exporting STIX 2.1

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/stix/feeds/misp/ips' > bundle.json
```

A namespace becomes a bundle shaped after the OASIS *Sighting of an Indicator* example: for each value an `indicator` carrying the pattern, and a `sighting` that refers to it with the count and the window.

```
{"type":"bundle","id":"bundle--...","objects":[
  {"type":"identity","id":"identity--...","name":"SightingDB", ...},
  {"type":"indicator","id":"indicator--...",
   "pattern":"[ipv4-addr:value = '198.51.100.23']","pattern_type":"stix",
   "object_marking_refs":["marking-definition--..."], ...},
  {"type":"sighting","id":"sighting--...",
   "count":5,"first_seen":"...","last_seen":"...",
   "sighting_of_ref":"indicator--...",
   "where_sighted_refs":["identity--..."],
   "x_sightingdb_namespace":"feeds/misp/ips"},
  {"type":"marking-definition","name":"TLP:AMBER", ...}]}
```

The namespace travels as `x_sightingdb_namespace`, which is a custom property and so carries the `x_` prefix the specification requires.

11.1.1 Ids are derived, not minted

Minting an id is creating a fresh one. A STIX identifier is the object's type, two dashes, and a UUID:

```
indicator--9299f726-ce06-492e-8472-2b52ccb53191
```

Normally that UUID is a **version 4** — random. Minting one takes nothing but a source of randomness, which is why it is the default: no agreement is needed, and two parties will

never collide. The cost is that it is new every time. Export the same value twice and you get two unrelated ids, and nothing downstream can tell they describe the same thing.

A **version 5** UUID is the other kind. It is a hash of two inputs:

- a **namespace UUID**, chosen once and kept forever, and
- a **name** — any string, as long as you can produce the same one again.

Same two inputs, same UUID, on any machine, in any language, for ever. So "what it takes to mint" is really the question of which of those two you are doing: with v4, a random number; with v5, a namespace you have fixed and a name you can reconstruct.

SightingDB uses v5 everywhere, with a namespace of its own:

```
3ef11775-6174-4718-92bf-16705b732650
```

and these names:

Object	The name that is hashed
indicator	`indicator:<observable type>:<value>`
sighting	`sighting:<namespace>:<observable type>:<value>`
identity	`identity:<name>`
bundle	`bundle:<namespaces, comma separated>`

Which means you can mint the same ids yourself, without this server:

```
import uuid

SIGHTINGDB = uuid.UUID("3ef11775-6174-4718-92bf-16705b732650")

value, observable, namespace = "evil.example", "domain-name", "feeds/misp/domains"

print("indicator--" + str(uuid.uuid5(SIGHTINGDB, f"indicator:{observable}:{value}")))
print("sighting--"  + str(uuid.uuid5(SIGHTINGDB, f"sighting:{namespace}:{observable}:{value}")))
```

Those are exactly the ids the export produces for that value in that namespace — not an illustration of the idea, the same two strings hashed the same way. And they will not change between releases: the namespace above is a constant precisely so that they cannot.

Note which name carries what. The indicator's has only the observable type and the value in it, so the same domain seen in four namespaces is one indicator. The sighting's has the namespace as well, so each of those four is its own sighting of it.

> The specification asks for v4 on objects like indicators and sightings, and *requires* v5 for cyber-observable objects, with a namespace it fixes itself. Using v5 everywhere is therefore a deliberate departure: it trades the guarantee that two publishers never collide for the guarantee that the same data always produces the same bundle. For a database whose job is to be re-exported on a schedule, that is the better trade — and the namespace being ours is what keeps the collision risk where v4 would have put it.

Two consequences, both useful:

- Exporting the same data twice produces a **byte-identical** bundle, so an export can be diffed, cached, or published on a schedule without looking like it changed.
- One value in two namespaces yields **one indicator with two sightings**, because the indicator's name has only the value in it and the sighting's has the namespace too.

A value that arrived *as* STIX is the exception: the importer records the id its publisher gave it as a `stix-id:` tag, and the export uses that rather than deriving one. A bundle that goes out of here and comes back refers to the same objects it did before.

11.1.2 What decides the pattern

An indicator is a pattern, and there is no pattern without a type. The type is resolved in this order:

1. A `stix-type:` tag on the value.
2. The `[stix.types]` mapping for the namespace, read backwards.
3. The shape of the value itself.

A value that none of those identify is **skipped** and counted in `X-SightingDB-Skipped`, never guessed at. If you would rather have it, ask:

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/stix/feeds/misp/ips?untyped=include'
```

Those go out as a custom observable type, flagged, so a consumer can tell them from a value whose type is actually known.

11.1.3 Headers say what you got

Header	Meaning
`X-SightingDB-Exported`	Values in the bundle.
`X-SightingDB-Skipped`	Values left out for having no observable type.
`X-SightingDB-Untyped`	Of those exported, how many went out untyped.
`X-SightingDB-Namespaces`	How many namespaces contributed.
`X-SightingDB-Truncated`	The namespace holds more than `limit` allowed.
`X-SightingDB-Missing`	Namespaces asked for that do not exist.

A bundle that quietly dropped a third of a namespace would be worse than one that refused, so it says.

11.1.4 A subtree, or several namespaces

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/stix/feeds?recursive'
```

```
curl -H 'Authorization: changeme' -H 'Content-Type: application/json' \
  -X POST http://localhost:9999/_api/stix \
  -d '{"namespaces":["feeds/misp/ips","watchlist/ips"],"q":"198.51","limit":500}'
```

Through a load balancer, each namespace is exported by the mirror that holds it and the bundles are folded into one. The result is byte-identical to what a single server holding all of them would produce — the derived ids make the merge a deduplication rather than a reconciliation.

11.1.5 Configuring who you are

```
[stix]
identity = "Beta Cyber Intelligence Company"
identity_class = "organization"

[stix.types]
"feeds/misp/ips" = "ipv4-addr"
"feeds/misp/domains" = "domain-name"
"feeds/misp/hashes" = "file.SHA-256"
```

`identity` is who the bundle is published by and who the sightings are attributed to when no `identity:` tag says otherwise. `[stix.types]` saves tagging every value in a namespace that only ever holds one kind of thing.

> Verify changes against the OASIS library rather than by eye: `pip install stix2`, then `stix2.parse(raw, allow_custom=True, version="2.1")`. `allow_custom` is needed only for `x_sightingdb_namespace`.

11.2 Importing STIX

```
sightingdb --import-stix ./bundles/
```

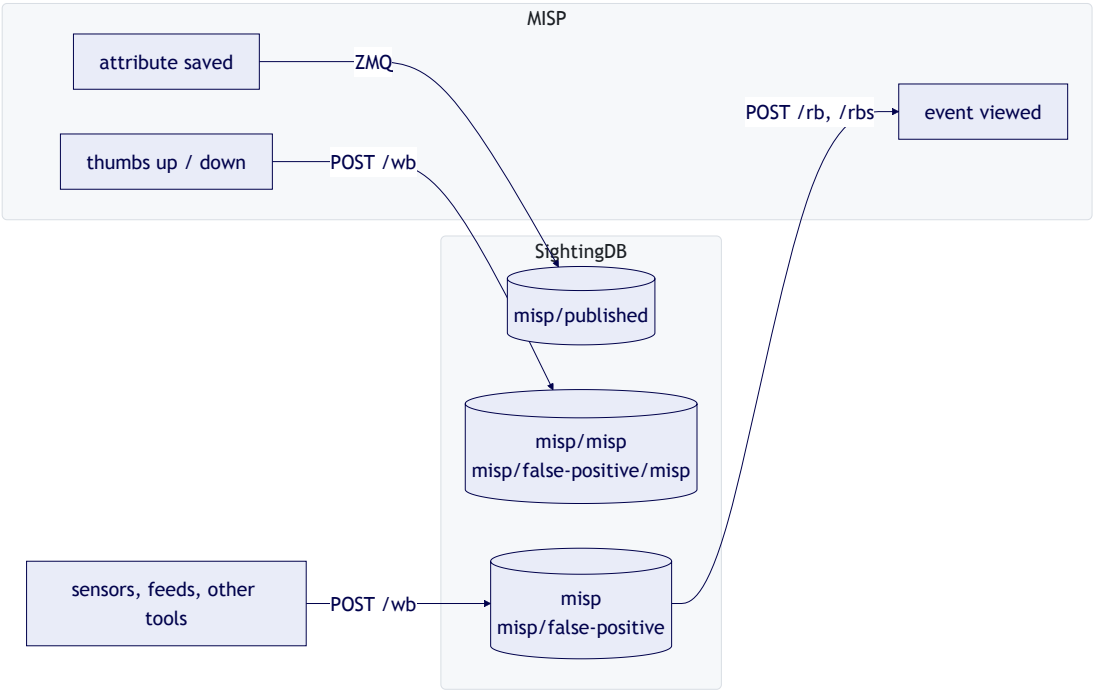
A file or a directory. It loads, writes, saves and exits — no listeners start, so it is safe to run against a server's data directory while that server is stopped.

A STIX sighting's `count` is honoured, and its `first_seen`/`last_seen` become the window: the first write lands at `first_seen` and the rest at `last_seen`, so both ends survive. Tags are written from what the bundle says — the observable type, the markings, who published it — which is what makes an import and then an export round-trip.

11.3 SightingDB and MISP

SightingDB feeds MISP's own **sightings** and **false-positives**: what it has counted for a value is added to the counts MISP shows for that attribute, as one more source beside the organisations that reported it, and drawn beside MISP's own in the sightings graph. Everything travels as the SHA-256 of the value — MISP looks values up by their digest — so a SightingDB that holds only digests still gives MISP everything it needs, and holds nothing that reveals what was seen.

Each kind of data has a namespace of its own. With a connection whose namespace is `misp`:



Namespace	Written by	Read by MISP as
`misp`	Whatever sees things: sensors, feeds, scripts	Sightings
`misp/false-positive`	The same, for values that turned out harmless	False-positives
`misp/misp`, `misp/false-positive/misp`	MISP, when an analyst records one	—
`misp/published`	MISP's ZMQ feed of saved attributes	—

MISP reads exactly two namespaces and writes to others. Lookups are exact — `misp` does not include `misp/misp` — so MISP never counts its own sightings a second time, and an attribute merely being saved in MISP is never mistaken for something having been seen. Everything else using SightingDB sees all of it.

The connection settings below — the false-positive namespace, pushing, the API key — need a MISP recent enough to have those fields. An older MISP shows SightingDB's sightings only in a column of their own, ignores false-positives, never writes, and sends no key.

11.3.1 The connection in MISP

Enable the integration under *Administration* → *Server Settings & Maintenance* → *Plugin settings*:

`Plugin.Sightings_sighting_db_enable = true`

and add the server under *Sync Actions* → *SightingDB* → *Add SightingDB connection*:

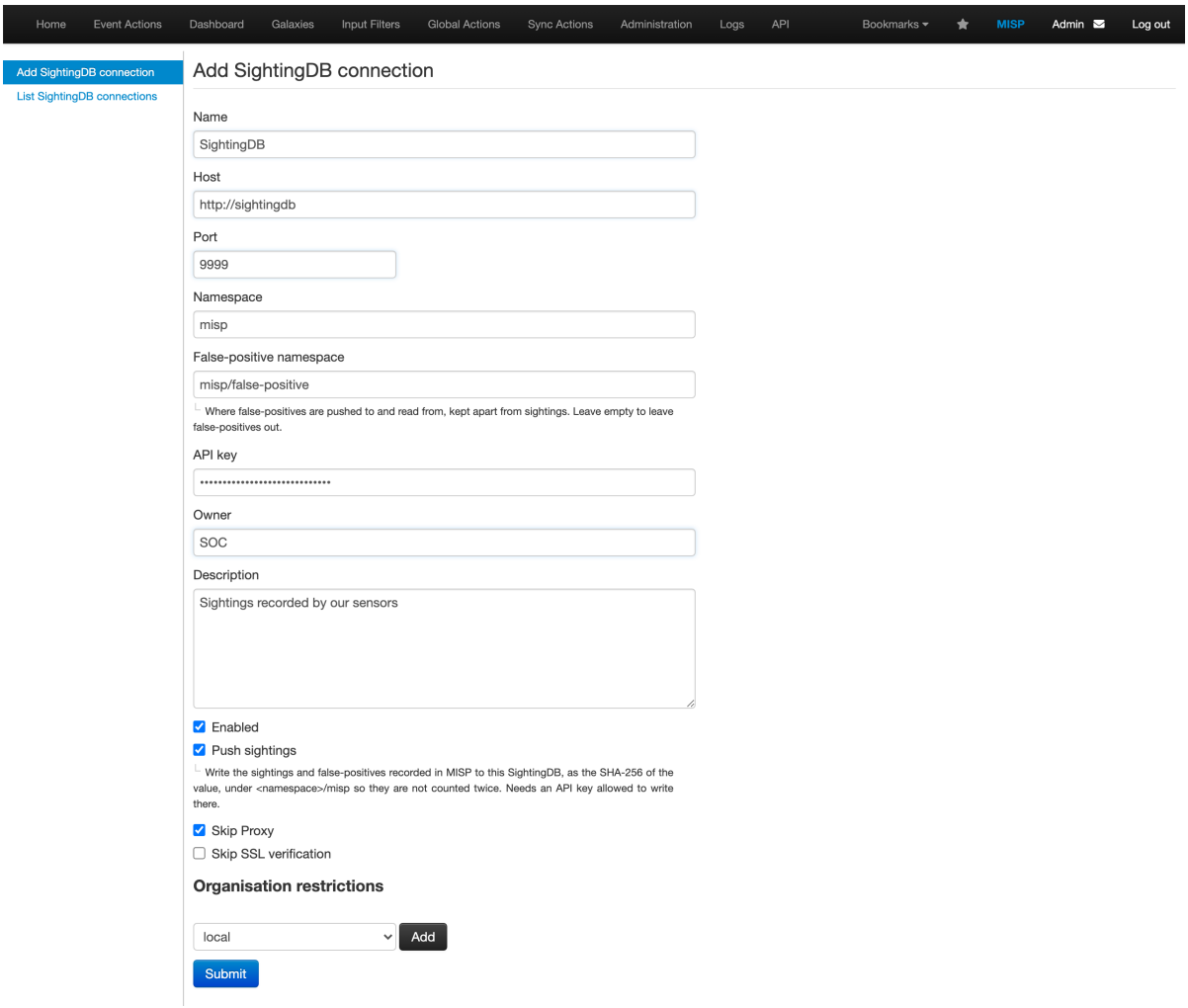


Figure 11.1: Adding a connection.

Field	What to put there
Name	How the server is named in the popover, the column and the graph.
Host	With its scheme: `http://sightingdb` or `https://sightingdb.example`. No trailing slash, no port.
Port	`9999` unless `listen_port` says otherwise.
Namespace	Where sightings are read from. Left empty, MISP uses a namespace literally called `all`.

Field	What to put there
False-positive namespace	Where false-positives are read from. Filled in as `<namespace>/false-positive` while you type the namespace; empty leaves false-positives out.
API key	Sent as the `Authorization` header. Leave it empty only for a server running with `authenticate = false`. It is never shown again; leaving it empty when editing keeps the stored one.
Owner, Description	Shown in the SightingDB column's details.
Enabled	Nothing is queried until it is.
Push sightings	Write the sightings and false-positives recorded in MISP to this server, under `/misp` below each namespace.
Skip Proxy	Tick it when SightingDB is on the same network as MISP and MISP has a proxy configured.
Skip SSL verification	For a self-signed certificate. Prefer giving MISP the CA.
Organisation restrictions	Left empty, every organisation sees the counts.

One key covers it all, reading and — for pushing — writing, since grants are by prefix. If the server keeps its keys in `acl_file`, create it in the interface (chapter 10); otherwise:

```
[acl]
"misp" = "rw:misp"     # "r:misp" if MISP only reads
```

The connection list has a **Run** button per connection, which asks the server who it is:

11.3.2 What MISP shows

The sightings cell of each attribute counts SightingDB's sightings and false-positives with MISP's own, and its popover names SightingDB as one more source:

A TTL is shown as an expiration. A value written with a `ttl` stops being valid that many seconds after its last sighting, which is what an expiration means in MISP. So when the value in the sightings namespace has one, MISP counts SightingDB among the expirations and shows the date, `last_seen` plus the TTL, beside any organisation's. A new sighting moves `last_seen`, and the date with it.

To see SightingDB on its own, press **SightingDB** in the attribute toolbar. A column appears with one line per server that knew the value, as `Name: count`, followed by its false-positives in red when it has any; hovering it shows when the value was first and last seen, its last false-positive, and when it expires:

The sightings graph — the wrench next to an attribute's thumbs, or the one on the event — keeps MISP's Sighting and False-positive lines for MISP's own, and draws SightingDB's beside

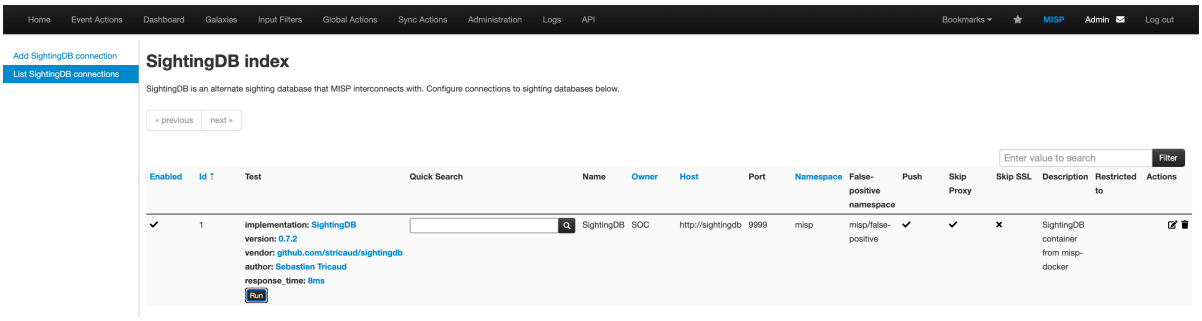


Figure 11.2: A tested connection.

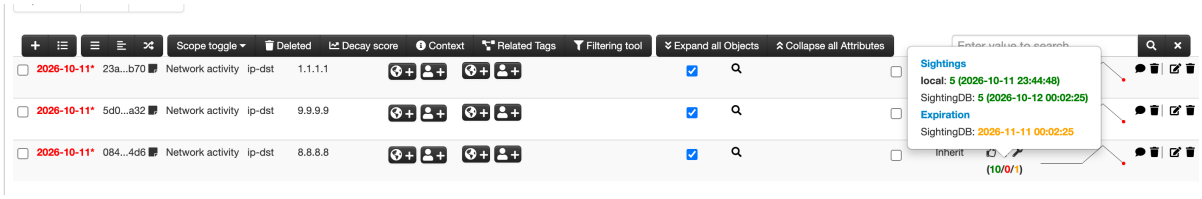


Figure 11.3: Sightings: MISP's own, and SightingDB's, with the date SightingDB will expire the value.

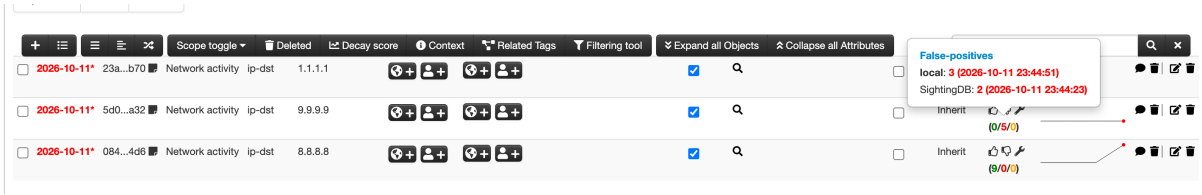


Figure 11.4: False-positives, the same way.

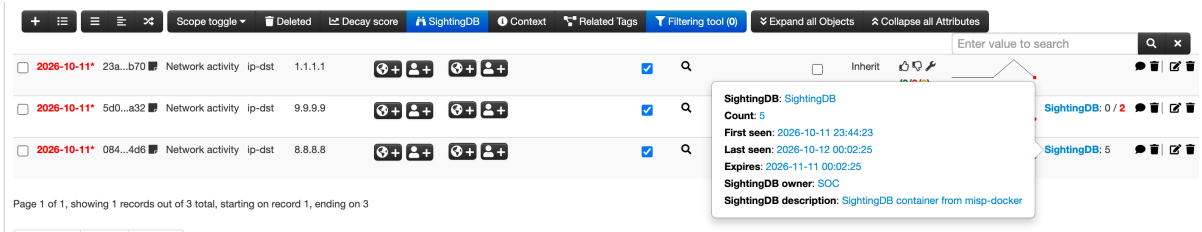


Figure 11.5: The SightingDB column, with a hit's details.

them, from the hourly statistics it keeps (`stats_retention`, chapter 4): one line for its sightings, and one for its false-positives when the connection has a false-positive namespace.

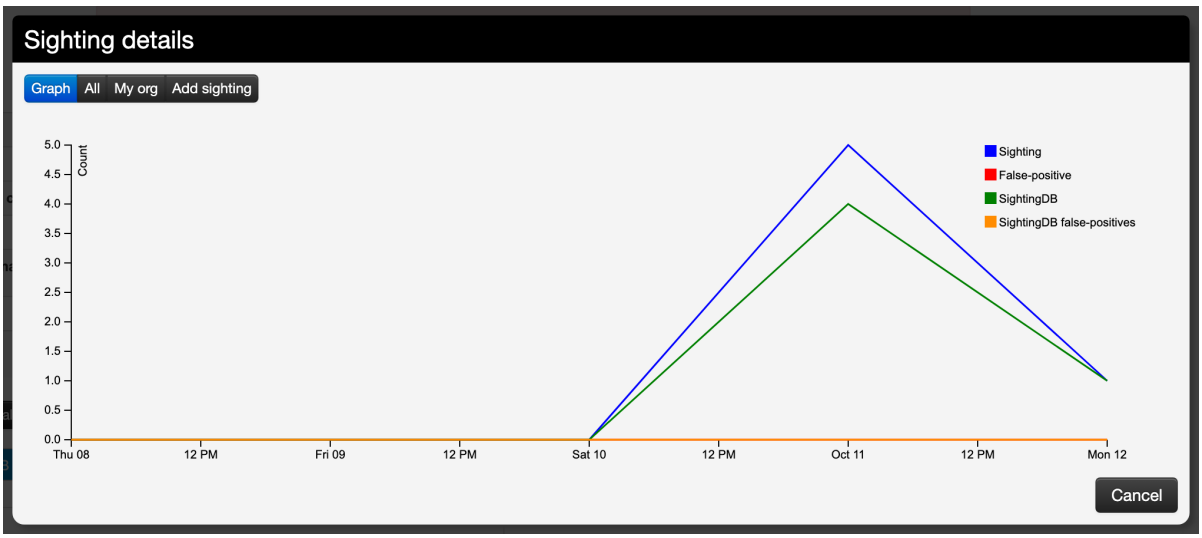


Figure 11.6: An attribute's sightings over time: MISP's, and SightingDB's.

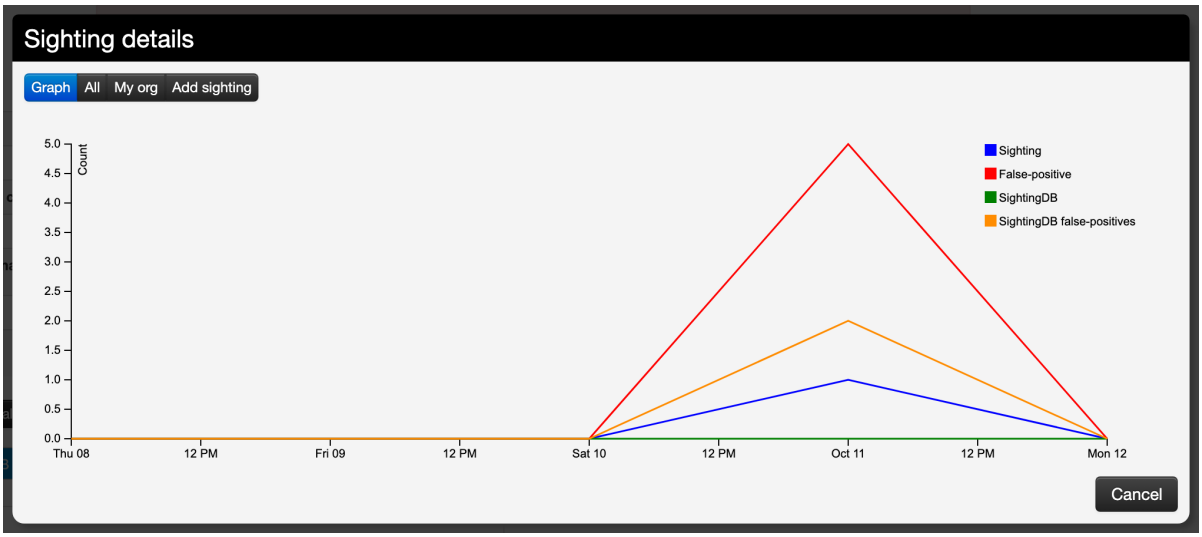


Figure 11.7: A value reported harmless, by MISP's analysts and by SightingDB's sources.

The lookups behind the counts send every attribute of the page in one `POST /rb` per connection, sightings and false-positives together, without `noshadow` — so each view of an event is recorded under `_shadow/<namespace>`, which is how you can tell which values your analysts keep looking at. They are given three seconds; a SightingDB that is down or slow costs the page that and no more. The graph's `POST /rbs` sets `noshadow`: drawing a graph is not someone looking a value up.

11.3.3 Feeding it

Anything that sees a value records it as its digest, in the namespace MISP reads:

```
value=8.8.8.8
digest=$(printf %s "$value" | sha256sum | cut -d' ' -f1)
curl -H 'Authorization: changeme' "http://localhost:9999/w/misp?val=$digest"
curl -H 'Authorization: changeme' "http://localhost:9999/w/misp/false-positive?val=$digest"
```

The digest is of the attribute's whole value, as MISP holds it: `1.2.3.4|80` is hashed as one string.

11.3.4 Pushing: what analysts record in MISP

With **Push sightings** ticked, the thumbs-up on an attribute — or a sighting added through MISP's API — is written to `misp/misp`, and the thumbs-down to `misp/false-positive/misp`, as the digest with the time it was recorded. It happens as the sighting is saved, with a five-second limit: a SightingDB that is down or slow never stops MISP saving the sighting, and the failure goes to MISP's error log.

Expirations are not pushed. In MISP an expiration is a judgement that an indicator is no longer valid; nothing is deleted. The nearest thing SightingDB has is a TTL, and an expired TTL is swept away — the value and its history gone, for every user of the namespace. Turning one into the other would make deciding an indicator is stale the same as erasing that it was ever seen.

MISP does not read these back: MISP-to-MISP sharing of sightings is what MISP synchronisation is for. They are there for everything else that uses SightingDB — and an instance that should count another instance's analysts as a source can point a second connection at that instance's push namespace.

11.3.5 Publishing: MISP's attributes over ZMQ

MISP publishes over ZMQ, and SightingDB subscribes. On the MISP side, under *Plugin settings*:

```
Plugin.ZeroMQ_enable = true
Plugin.ZeroMQ_host = 0.0.0.0      # 127.0.0.1 is unreachable from another host or container
Plugin.ZeroMQ_attribute_notifications_enable = true
Plugin.ZeroMQ_event_notifications_enable = true
```

On SightingDB's, keeping digests only, in a namespace of their own:

```
[zmq]
endpoint = "tcp://misp.example:50000"
format = "misp"
topics = ["misp_json_attribute", "misp_json"]
require_to_ids = false
hashed_namespace = "misp/published"
store_values = false
```

`hashed_namespace` records the SHA-256 of every ingested attribute — mapped type or not — into that one namespace, which then holds a digest of every value MISP has, with a count of how often it was published. Keep it out of the namespaces MISP reads: an attribute being saved is not a sighting. `store_values = false` skips the value itself, so `[zmq.types]` has nothing

to do and can be left out.

To keep the values as well, for the interface and the STIX export, leave `store_values` at its default and map types as usual:

```toml
[zmq]
endpoint = "tcp://misp.example:50000"
format = "misp"
default_namespace = "misp/other"
require_to_ids = false
hashed_namespace = "misp/published"

[zmq.types]
ip-src = "misp/ips"
ip-dst = "misp/ips"
domain = "misp/domains"
hostname = "misp/domains"
url = "misp/urls"
md5 = "misp/hashes"
sha256 = "misp/hashes"
```

Attributes are read from single-attribute publications and from whole events, including attributes nested inside objects — which is where most of a modern MISP event's attributes live. Only mapped types are ingested as values unless `default_namespace` is set, and `require_to_ids = true` limits ingest — values and digests alike — to attributes MISP flagged as actionable.

MISP's timestamps are preserved, so a replayed event does not claim to have been seen today. SightingDB logs each subscription as it connects — `ZMQ ingest connected to tcp://misp.example:50000 (2 topic(s))` — and retries every `reconnect` seconds until it can.

What becomes a tag

Tag	From	On a digest
`misp-type:`	The attribute type, as published.	Kept
`stix-type:`	Translated from it, where there is a one-to-one mapping.	Dropped: the digest is not of that type
`misp-category:`	The attribute's category.	Kept
`misp-event:`	The event it came from.	Dropped: it leads back to the value
`description:`	The attribute's comment, if it has one.	Dropped: comments often repeat the value
everything else	MISP's own tags, carried across as published.	Kept

MISP's tags are already `namespace:predicate` shaped — `tlp:amber`, `misp-galaxy:threat-actor="Callisto"` — so they come across unchanged, which is what makes `tlp:` work end to end. A free-form

tag that happens to contain the value cannot be told apart from any other, and is carried to the digest too.

Tags on the event are inherited by its attributes. MISP keeps the TLP marking on the event, and reading attribute tags alone would lose it: a sighting from a `tlp:amber` event would arrive unmarked and export unmarked, which is the one kind of loss here that could mislead someone about how a value may be shared.

Where the two contradict each other the attribute wins and the event's is dropped rather than added beside it — a value tagged both `tlp:amber` and `tlp:white` says nothing useful. That applies only to taxonomies where one value can be true at a time, which today means `tlp:`; most are genuinely multi-valued, so an event and an attribute can each contribute a `misp-galaxy:` tag and both are kept.

11.3.6 Checking each path

Browsing `misp` in the interface shows the namespaces each path writes to, holding digests and never values:

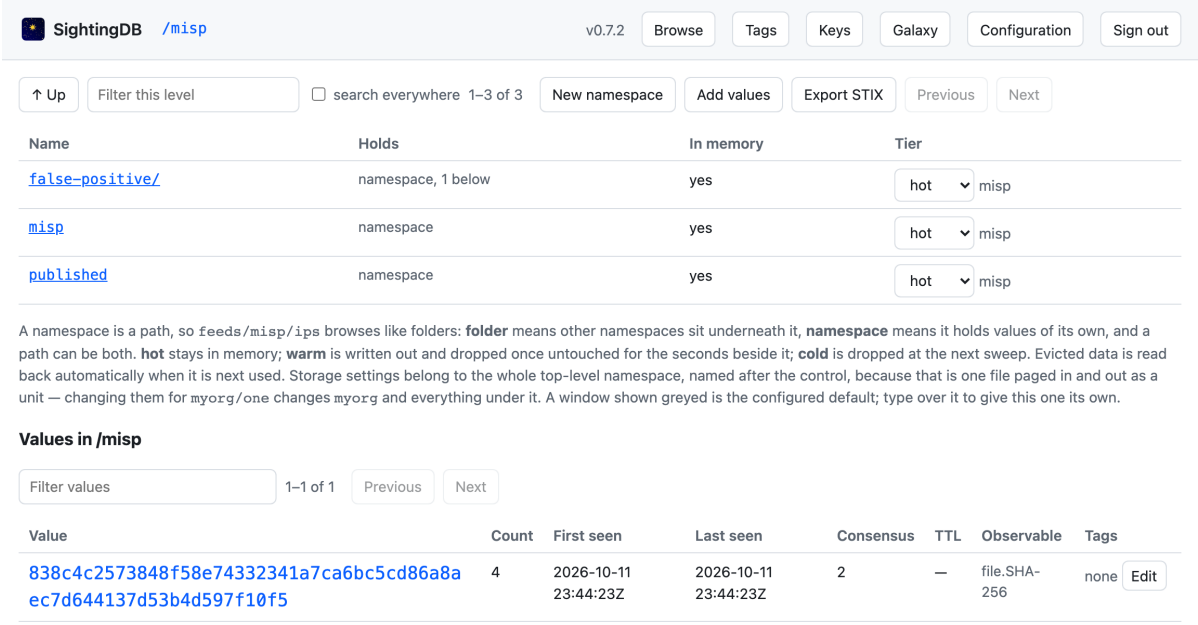


Figure 11.8: What MISP and its feeds wrote.

The **Observable** column guesses a value's type from its shape, and a 64-character hex string looks like a file's SHA-256. It is not one; it is the digest of whatever the attribute held.

With `d` standing for a value's digest, `printf %s 8.8.8.8 | sha256sum`:

To check	Do	Then
Reading	Record the digest in `misp` (above)	MISP's sightings count for the attribute goes up by one
False-positives	Record it in `misp/false-positive`	MISP's false-positive count goes up by one

To check	Do	Then
Pushing	Press the thumbs-up in MISP	`GET /r/misp/misp?val=<d>&noshadow`: `count` up by one, `last_seen` now
Publishing	Add an attribute in MISP	`GET /r/misp/published?val=<d>&noshadow`: `count` 1

11.4 A feed to try it with

CIRCL publish a public MISP OSINT feed at https://www.circl.lu/doc/misp/feed-osint/. The galaxy sample in `doc/docker` is loaded from one of its events, converted by `doc/docker/fetch-demo-data.py` into the shape this ingest produces — same namespaces, same tags. It is a useful thing to read if you are writing an importer of your own.

Two cautions if you take data from it yourself. The feed is **not uniformly public**: most events are `tlp:white` or `tlp:clear`, but some are `tlp:green` and a few carry no marking at all, and `tlp:green` means the community rather than the world. And it states **no licence** — TLP is a handling designation, not a copyright grant. The script that converts it refuses anything not marked for unrestricted sharing for exactly that reason.

12

Operations

12.1 Is it alive

```
curl -fsS http://localhost:9999/health
```

```
{"status":"ok","version":"0.7.1","uptime_seconds":864,
 "resident_shards":4,"shards":7,"catching_up":false}
```

No key, which is what makes it usable as a container healthcheck and a load balancer probe. `resident_shards` against `shards` is how much of the database is in memory; `catching_up` is true while a node is pulling what it missed.

12.2 Where the data is

Everything lives in memory and is written to `dbdir` as compressed JSON, one file per top-level namespace, every `snapshot_interval` seconds and once more on a clean shutdown.

That shape is the reason `dbdir` matters so much: **without it, nothing survives a restart.** The server will start, work perfectly, and lose everything when it stops.

A snapshot that exists but cannot be read is fatal at startup rather than ignored. Starting empty would look like catastrophic data loss, and the next save would make it real.

Beside the shards sits `activity.json`, the read and write counts behind the interface's Activity page (chapter 10). It is treated differently on purpose: an unreadable one is logged and ignored, because it describes how the database was used rather than what it holds, and starting without it loses nothing that matters.

Snapshots carry a format version and are migrated forward on load, so an upgrade reads what the previous version wrote. The migration happens in memory and lands on disk at the next save.

12.3 What stays in memory

Chapter 4 has the settings; this is what to do with them.

A top-level namespace is the unit. `hot` keeps it in memory, `warm` writes it out and drops it after

`warm_idle` seconds untouched, `cold` drops it at the next sweep. Evicted data is **read back when it is next used**, so `cold` costs one load per burst of activity rather than one per operation.

```
curl -H 'Authorization: changeme' -X POST http://localhost:9999/_api/tier \
  -H 'Content-Type: application/json' \
  -d '{"namespace":"archive","tier":"warm","warm_idle":86400}'
```

```
{"shard":"archive","tier":"warm","warm_idle":86400,
 "own_tier":true,"own_warm_idle":true,
 "effect":"'archive' and everything under it is dropped after 86400s untouched"}
```

The reply says the effect out loud, because a change made from a row deep in a tree is a change to everything beside it. Either field takes `"default"` to stop overriding.

A rough guide: feeds you enrich against are `hot`; feeds you keep for history are `warm` with a long window; anything you query a few times a month is `cold`.

12.4 Expiring data

A `ttl` on a write is how long the value stays visible. The sweep removes what has expired, every `sweep_interval` seconds, and gives back the consensus those values were holding.

Shadow sightings — the record of what was searched for — have their own `shadow_ttl`, 30 days by default, because "what did people look up" is interesting for a while and not forever.

12.5 What was refused

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/_management/api/rejections'
```

Newest first, from every write path including the ZMQ ingest, which has no caller of its own to tell. This is the answer to "the feed says it sent fifty thousand and I have forty-nine thousand".

Bounded and in memory. A record kept to make something visible, not a fact about the data.

12.6 How busy each namespace is

```
curl -H 'Authorization: changeme' \
  'http://localhost:9999/_management/api/activity?granularity=day&buckets=7'
```

Reads and writes per top-level namespace, one count per bucket, with the totals for the window and for the window before it. `granularity` is `hour` or `day`, `end` is any Unix time in the last bucket wanted, and `buckets` is how many. It is what the Activity page draws, and it is as easy to feed into whatever else watches the server: a namespace whose reads drop to zero is a feed nobody consumes any more, and one whose writes drop to zero is an ingest that has stopped.

`activity = false` in `[daemon]` switches the counting off.

12.7 Certificates

```
[daemon]
ssl = true
ssl_cert = "/etc/sightingdb/tls/cert.pem"
ssl_key = "/etc/sightingdb/tls/key.pem"
```

`sightingdb --install-selfsigned-keys` writes a pair at those paths for getting started. The Configuration page shows the expiry and warns at 30 days.

That warning matters more in a galaxy than it looks: when a certificate lapses, peers stop being able to authenticate to each other, and a galaxy comes apart without any one server failing. For a galaxy of self-signed instances — which is what `--setup` produces — set `verify_tls = false` in `[galaxy]` and understand what you have chosen.

12.8 Logs

`log_out` and `log_err` in `[daemon]`, or stdout and stderr in a container. Every line is timestamped, including panics, which is worth knowing because a crash with no timestamp is hard to line up against anything else.

For more control, `-l` takes a log4rs configuration.

12.9 Backing up

Stop the server, or take the snapshot directory while it runs and accept that you get the last snapshot rather than the current state — up to `snapshot_interval` behind. The files are self-contained; copying `dbdir` copies the database.

The machine-owned files from chapter 4 — `acl.toml`, `tiers.toml`, `tags.toml`, `peers.toml` — are small, change rarely, and are worth having in the same backup. The keys in particular: a galaxy whose load balancer has lost its `peers.toml` has lost the credentials it reaches its nodes with.

12.10 Upgrading

One binary. Stop it, replace it, start it. Snapshots migrate forward on load, and the configuration you have keeps working — new settings have defaults that preserve the previous behaviour.

In a galaxy, upgrade the nodes before the routers. A galaxy running mixed versions is visible on the Galaxy page, which reports each peer's version, because that is worth seeing before it misbehaves rather than after.

Reload the management interface once after an upgrade if it looks wrong; see the end of chapter 10.

12.11 Things that have surprised people

Reads are recorded. A read without `noshadow` writes a shadow sighting. Enrichment loops should pass it; an analyst's lookup should not.

`consensus` **counts namespaces, not sightings.** A value seen four thousand times in one feed has a consensus of one.

Deleting removes a namespace, not a value. There is no delete-one-value route; use a `ttl`.

`node_id` **must be unique in a galaxy.** Two servers sharing one quietly corrupt the counting, and nothing will tell you.

A new node needs peers of its own, or reads hashed to it will miss. See chapter 9.

13

Reference

The short version of everything, for finding rather than reading.

13.1 Routes

Route	Method	What it does
`/w/<namespace>?val=`	GET	Record a sighting.
`/wb`	POST	Record many.
`/vwb`	POST	Check a batch without recording it.
`/r/<namespace>?val=`	GET	Read one value.
`/r/<namespace>`	GET	List a namespace.
`/r/<namespace>?count`	GET	How many values it holds.
`/rs/<namespace>?val=`	GET	Read with hourly statistics.
`/rb`, `/rbs`	POST	Read many, without and with statistics.
`/d/<namespace>`	GET	Delete a whole namespace.
`/stix/<namespace>`	GET	Export as a STIX 2.1 bundle.
`/_api/stix`	POST	Export several namespaces into one bundle.
`/_api/merge`	POST	Fold a peer's copy of values into ours.
`/_api/namespaces`	GET	What this server actually stores.
`/_api/tier`	POST	Set a namespace's tier.
`/_api/openapi.yaml`	GET	The whole API as OpenAPI 3.
`/health`	GET	Liveness. No key needed.
`/i`	GET	Version and implementation.
`/`	GET	The route list. No key needed.

Management, all under `/_management/api/`:

Route	Method	What it does
`session`	GET	Is this an admin key?
`info`	GET	What this server was configured to do.
`namespaces`	GET, POST	List, or create an empty one.
`tree`	GET	One level of the namespace tree.

Route	Method	What it does
`activity`	GET	Reads and writes per top-level namespace over time.
`values`, `value`	GET	Values in a namespace; one value.
`sightings`	GET	Where else a value has been seen.
`tags`	GET, POST	The vocabulary; replace a value's tags.
`tags/vocabulary`	POST, DELETE	Define or forget a tag's colour.
`galaxy`	GET	The whole cascade, for drawing.
`galaxy/peers`	GET, POST, PUT, DELETE	List, add, change, remove a peer.
`galaxy/peers/enabled`	POST	Take a peer out of service, or back.
`keys`	GET, POST, PUT	List, save, or replace the whole list.
`keys/generate`	GET	A random key.
`keys/drift`	GET	Where this server's keys and its peers' disagree.
`keys/<key>`	DELETE	Remove a key.
`rejections`	GET, DELETE	What was refused; forget it.
`tier`	POST	Set a tier.

`doc/routes.md` has every one of these with a worked example and its status codes. A test keeps it from drifting.

13.2 Query parameters

Parameter	On	Effect
`val=`	`/w`, `/r`, `/rs`	The value.
`noshadow`	reads	Do not record that somebody searched.
`timestamp=`	`/w`	Unix seconds; record it as seen then.
`ttl=`	`/w`	Seconds until it stops being visible.
`tags=`	`/w`	Tags to merge in.
`count`	`/r`	Answer with the number of values instead.
`for_merge`	`/r`	The per-node shape a peer is offered.
`recursive`	`/stix`	Include the namespaces below this one.
`untyped=include`	`/stix`	Export values with no observable type.
`limit=`, `q=`	listings, `/stix`	Page size; substring filter.

13.3 Status codes

Code	Meaning
200	Done. A batch may be `partial`; read `items`.
400	Malformed, or a batch that failed for mixed reasons.
401	No key.
403	Not permitted, or a write to an internal namespace.
404	No such namespace, or no such value in it.
409	Refused because of how things are configured.
421	Nowhere in reach stores that namespace.
500	Something broke.
502	A mirror that holds it could not be reached.
508	A forwarded request ran out of hops.

13.4 Configuration at a glance

Section	Setting	Default
`[daemon]`	`listen_ip`, `listen_port`	`127.0.0.1`, `9999`
	`authenticate`	`true`
	`ssl`, `ssl_cert`, `ssl_key`	off
	`dbdir`	none — **nothing persists without it**
	`snapshot_interval`	`300`
	`sweep_interval`	`60`
	`stats_retention`	`720` hours
	`shadow_ttl`	`2592000` seconds
	`post_limit`	2.5 GB
	`rejection_log`	`1000`
	`activity`	`true`
	`node_id`	`"local"` — **unique per server in a galaxy**
	`acl_file`, `tags_file`	none
`[storage]`	`namespaces`	everything
	`default_tier`, `warm_idle`	`hot`, `3600`
	`tiers_file`	none
`[galaxy]`	`peers`, `peers_file`	none
	`max_hops`	`4`
	`health_interval`	`30`
	`sync_interval`	`300`
	`reconcile_interval`	`3600`
	`gossip_interval`	`600`
	`acl_authority`, `acl_replaceable`	`false`
	`verify_tls`	`true`
`[stix]`	`identity`, `identity_class`	`SightingDB`, `system`
`[zmq]`	`url`, `format`, `types`	none
`[dns]`	`listen`, `zone`, `ttl`	none

13.5 The files

File	Owner	Rewritten when
`sightingdb.toml`	you	never
`acl_file`	the program	a key is saved
`tiers_file`	the program	a tier changes
`tags_file`	the program	a tag's colour changes
`peers_file`	the program	a peer is added, changed or disabled

13.6 Internal namespaces

Namespace	What it is	Readable	Writable
`_all`	The consensus tally.	yes	**no**
`_shadow/<ns>`	What was searched for in `<ns>`.	yes	no
`_config`	Server state.	no	no

Nothing beginning with `_` is writable from outside, at any permission level. A client that could write `_all` could forge consensus.

13.7 The Python client

Call	Does
`write(ns, value, timestamp=, ttl=)`	One sighting. Returns the count.
`write_many(sightings, strict=)`	A batch.
`batch(chunk_size=)`	A self-flushing batch, as a context manager.
`read(ns, value, stats=, shadow=)`	One value.
`read_many(sightings, stats=)`	Many, positionally.
`exists(ns, value, shadow=)`	Has this ever been seen?
`list_values(ns)`	Every value in a namespace.
`delete(ns, missing_ok=)`	A whole namespace.
`info(), ping(), close()`	Version; liveness; done.

`AsyncSightingDB` is the same with `await`.

13.8 Where else to look

`README.md`	The repository's own tour, in more depth on some things.
`doc/routes.md`	Every route, with examples and status codes.
`doc/ADDING-NEW-NODE.md`	Joining a node to a galaxy, with troubleshooting.
`doc/docker/`	A four-container galaxy to try things on.

`doc/openapi.yaml`	The API as a machine-readable document.
`etc/sightingdb.toml`	Every setting, with the reasoning beside it.